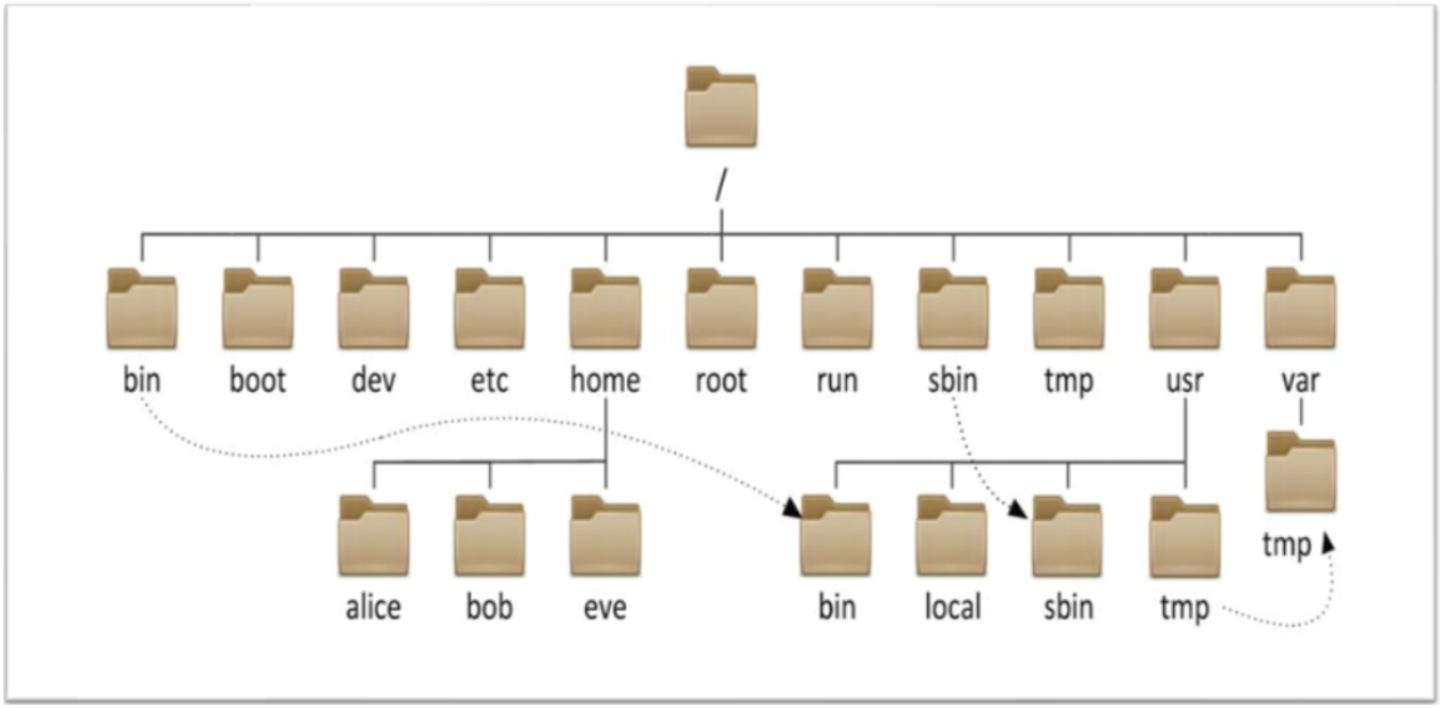
阶段二：Linux运维核心命令

今日内容：

- 1- 用户和用户组的管理： 用户相关操作
- 2- 权限与所有权管理：
  - 2.1 权限介绍
  - 2.2 设置文件或目录的权限
  - 2.3 特殊权限
  - 2.4 文件的所属用户和所属组设置
  - 2.5 ACL权限设置： 针对某个用户设置权限

1. Linux的目录结构

Linux的目录结构是一个树型结构，没有盘符这个概念, 只有一个根目录 /, 所有文件都在它下面，而不像Windows可以拥有多个盘符, 如 C盘、D盘、E盘



- Linux常见目录结构（高亮部分的要格外注意）

目录	作用
/bin	二进制命令所在的目录(普通命令 => 普通用户和超级管理员root)

/boot	系统引导程序所需要的文件目录，相当于Windows中的C盘
/dev	/device缩写，设备文件目录，磁盘，光驱 => /dev/sr0
/etc	系统配置文件目录，启动程序，几乎所有的软件都会把自己的配置文件安装在/etc中
/home	普通用户的家目录，默认用户数据存放目录，itheima账号 => /home/itheima文件夹
/lib	共享库文件和内核模块存放目录，软件安装、运行依赖库文件.a、.so文件
/mnt	临时挂载储存设备的挂载点，插入u盘、移动硬盘 => 先挂载 => /mnt中访问
/opt	额外的应用软件包， 安装qq、游戏、wps办公软件
/proc	process进程目录，操作系统运行时，进程信息和内核信息存放在这里
/root	Linux超级权限用户root的家目录，超级管理员root => /root
/sbin	和管理系统相关的命令，【超级管理员用】，s = super超级
/tmp	临时文件目录，这个目录被当作回收站使用
/usr	用户或系统软件应用程序目录，类似Windows中的Program files

总结说明:

```
家目录:
    /home:  普通用户的家目录，默认会在此目录下，根据用户名构建一个子目录，作为该用户家目录
    /root:  超级管理员root用户的家目录

用于放置Linux命令的目录:
    /bin :  用于放置操作Linux系统的普通命令 例如 创建文件 创建文件夹  查看文件 ....
    /sbin:  用于放置管理Linux系统的命令  例如: 重启服务器  关闭服务器

用于安装软件的位置:
    /opt: 一般在安装第三方软件的时候，通常放置到/opt目录下
    /usr: 一般是软件默认安装的位置

Linux系统各种配置文件目录: /etc

设备信息放置的目录: /dev

临时存储文件的目录: /tmp

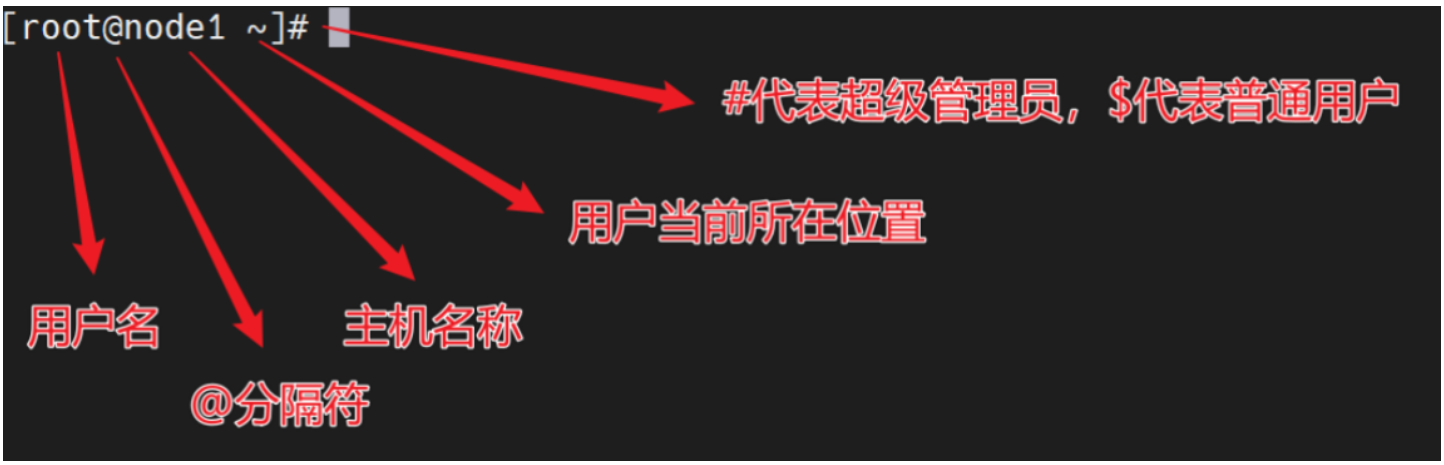

注意: 在Linux中目录结构是一个树形结构，没有盘符的概念，最顶级的为 / 表示根目录
```

2. Linux与目录相关命令

Linux目录的相关命令主要包括有pwd、ls、mkdir、cd、rmdir等， 如果把Linux的目录结构比作一个“文件柜”：

- `pwd` = 当前在文件柜的哪一层抽屉（显示当前位置）
- `ls` = 打开抽屉看看里面有什么（列出内容）
- `mkdir` = 增加了一个新的抽屉（创建目录）
- `cd` = 走进某个抽屉或文件夹（切换位置）
- `rmdir` = 扔掉一个空文件夹（删除空目录）

2.1 终端说明



2.2 基本导航命令

2.2.1 定位与查看（pwd、ls）

pwd（print working directory）：

- 格式： pwd
- 作用： 查看当前所在位置

```
[root@node1 ~]# pwd
/root
```

ls（list show）：

- 格式： ls [选项][路径]
- 作用： ls 是英文单词list show的简写, 其功能为列出目录的内容，是用户最常用的命令之一

ls常用选项

选项	含义
-a	all所有, 显示指定目录下所有子目录与文件, 包含隐藏文件
-l	list，以列表方式显示文件的详细信息
-h	配合 -l 以人性化的方式显示文件大小（文件大小 + 单位）

ls -l与ll显示信息说明：



2.2.2 切换与目录结构（cd、tree）

cd（change directory）：

- 作用：更改当前的工作目录
- 格式：

命令	含义
cd	切换到用户主目录（root用户主目录是/root,其他用户是/home/用户名）等价于 cd ~
cd 目录	切换到指定目录下 => cd /etc
cd ..	切换到上级目录

```
普及：路径有两种写法

绝对路径 ：从根目录一级一级向下移动，不能越级。如/home/itheima
例如：访问根目录下的usr目录下local目录下的nginx文件夹
cd /usr/local/nginx => 路径不需要记忆，用到的时候，直接按Tab键（自动补全）

相对路径 ：顾名思义，有一个参考点 => 以当前位置作为参考
```

tree：树形结构可视化辅助工具

- 安装 `tree` 命令

```
dnf -y install tree
# 查看目录结构
tree game_saves
```

输出示例:

```
game_saves
├── level1
├── level2
└── backup
    └── 2024
```

2.3 目录管理命令

2.3.1 创建目录（mkdir）

mkdir（make directory）：

- 作用：mkdir 命令用于创建目录
- 格式：

```
mkdir [-p] dirName

参数:
-p：一次创建多级目录
```

2.3.2 删除空目录（rmdir）【了解】

作用：rmdir命令用于删除空目录

```
rmdir dirName
```

清屏命令： ctrl + l

2.4 综合练习

2.4.1 搭建一个【游戏存档】目录

任务：

1. 在根目录创建 `game_saves` 目录，进入它
2. 创建子目录 `level1`、`level2`、`backup`
3. 在 `backup` 中创建 `2023` 和 `2024` 目录
4. 删除 `backup/2023`，再尝试删除整个 `game_saves`（观察错误）

2.4.2 搭建一个family目录

结构要求：

```
family
├── parents
│   ├── dad
│   └── mom
└── children
    ├── son
    └── daughter
```

要求：用 `tree` 命令验证结构，最后删除整个目录（只用已学命令）。

3. Linux与文件相关命令

3.1 文件操作命令

3.1.1 创建文件（touch）

作用：touch命令创建新文件

格式：touch 文件名

案例演示：

```
需求一：  在家目录下，创建一个hello.java
cd
touch hello.java

需求二：  在家目录下，创建一个a.txt
cd
touch a.txt
```

> 注：touch命令不仅可以创建.txt文本文档，还可以创建python/java/go等脚本程序。如`touch main.py`

3.1.2 复制 cp

作用：cp（copy）命令用来实现文件或者目录的复制

格式：cp [-r] 源文件位置... 目标路径

参数	说明
-r	递归,复制文件夹必须添加

```
需求一： 分别将家目录下的hello.java 和 a.txt 拷贝到 /usr/local
cd
cp hello.java /usr/local
cp a.txt /usr/local
或者
cp hello.java a.txt /usr/local

需求二： 将/usr/local 这个目录 拷贝到家目录下
cd /usr
cp -r local /root
cp -r local ~
```

3.1.3 移动 mv

作用：mv（move）命令用于文件、目录的移动和重命名

格式：mv 源文件路径... 目标路径

> 注意：mv没有任何选项，移动文件和文件夹都可以

案例演示：

```
需求一：将 /root目录下的anaconda-ks.cfg文件,移动到 /export/data 目录下
mkdir -p /export/data
cd ~
mv anaconda-ks.cfg /export/data

需求二：将 /export目录移动到 /root目录下
cd /
mv export ~
```

重命名：

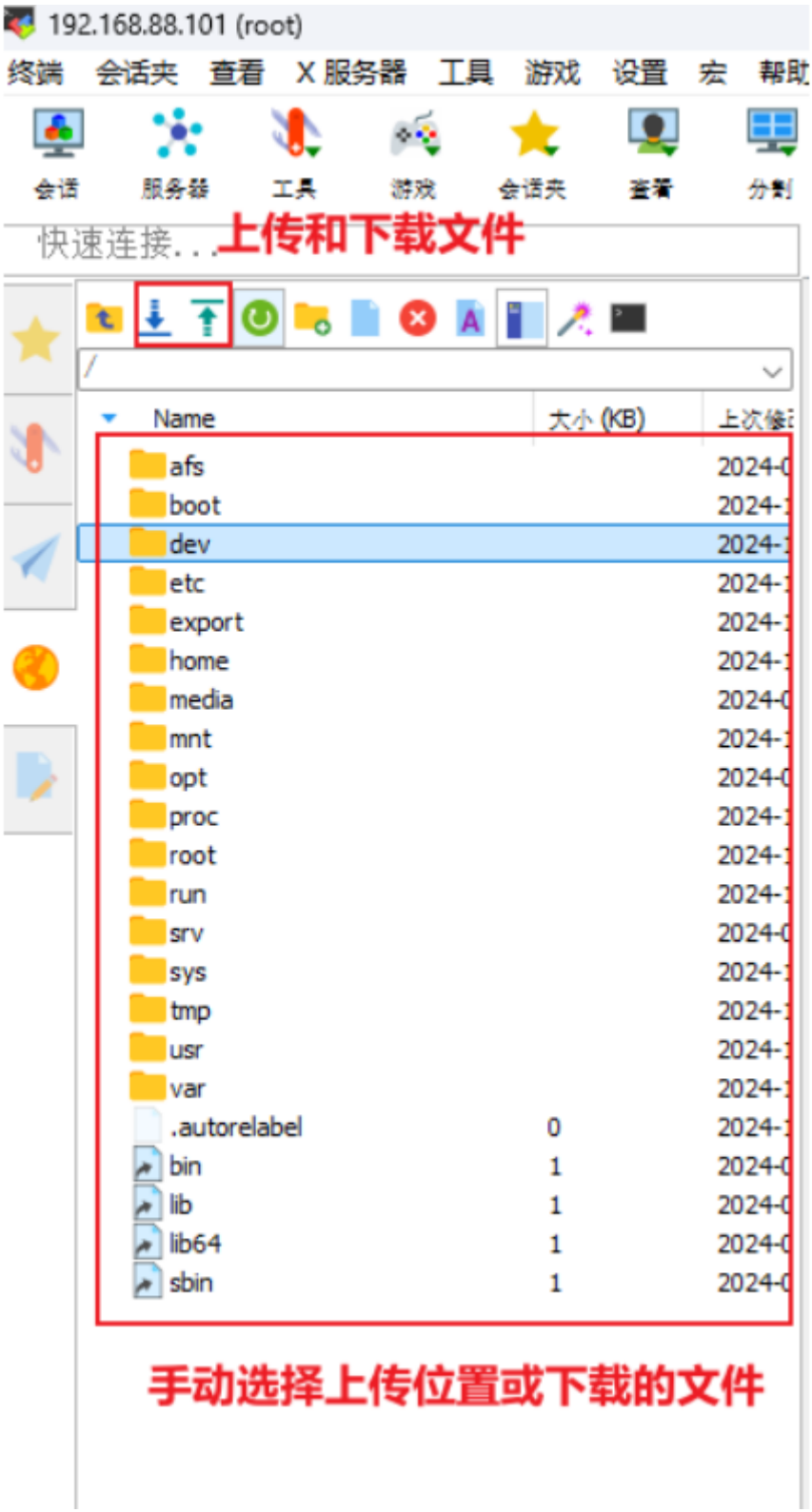
```
需求：将root用户家目录的a.txt文件，更名为 test.txt
cd ~
mv a.txt test.txt

需求二：将test.txt 移动到 / 目录下,并改名 a.txt
cd ~
mv test.txt /
cd /
mv test.txt a.txt

或者
cd ~
mv test.txt /a.txt


```

3.1.4 文件上传与下载



3.1.5 删除 rm

格式：rm [选项] 文件/目录路径

选项	说明
-r	是否启动递归删除方案（适用于删除目录的时候）
-f	是否需要强制删除，如果不添加， 会有一个询问的操作

3.1.6 文本查找相关命令 find

作用: 用于查找文档

语法: find 路径范围 选项1 选项1的值 [选项2 选项2 的值…]

选项	说明
-name	按照文档名称进行搜索（支持模糊搜索） * : 通配符，匹配任意个任意字符
-type	按照文档的类型进行搜索，文档类型的值，f（file)表示文件，d(directory)表示文件夹
-size	用于根据文件大小查找文件。格式为 `[+/-]n[单位]` `+` 和 `-` 前缀: • `+n`：查找大于 `n` 的文件。 • `-n`：查找小于 `n` 的文件。 单位说明: • `c`：字节（默认单位） • `k`：千字节（KB） • `M`：兆字节（MB） • `G`：吉字节（GB） • `b`：512字节块（不常用）
-mtime	用于根据文件的修改时间（modification time）来筛选文件,参数值是一个整数，表示天数 • `-mtime +n`：查找修改时间超过 `n` 天前的文件 • `-mtime -n`：查找修改时间在 `n` 天以内的文件 • `-mtime n`：查找修改时间正好是 `n` 天前的文件
-exec	用于对 `find` 查找到的文件执行指定的命令. 它的语法比较特殊，需要用 `{}` 表示查找到的文件，并用 `\;` 结束命令 例如: 查找所有 `.log` 文件并删除 find /var/log -name "*.log" -exec rm -f {} \;

```
# 查询 在/usr/lib目录下大于100M的文件
find /usr/lib -size +100M -type f

# 查询 在 /var/log下面 所有以 .log 结尾的文件
find /var/log -name "*.log" -type f
```

```
6
7
8   # 查询 在 /var/log下面 所有以 .log 结尾的文件 并且文件大于10kb的
9   find /var/log -name "*.log" -size +10K -type f
10
11  # 查询系统中，大于30天的文件或目录
12  find / -mtime +30
13
14  # 查询 在 /var/log下面 所有以 .log 结尾的文件 并且文件大于10kb的  将查询到的内容执行
    删除操作
15
16  find /var/log -name "*.log" -size +10K -type f -exec rm -rf {} \;
17
18  # 查询 在/usr/lib目录下大于100M的文件 ，  将并查询到的结果复制到/root目录下
19  find /usr/lib -size +100M -type f -exec cp {} /root \;
```

3.2 文件操作命令综合练习

3.2.1 日志文件管理与备份

某服务器日志目录 /var/log/app 需要定期清理和备份。要求：

- 创建 6 个不同日期的日志文件（如 access_20250225.log、access_20250302.log ...）
  - 说明: 如何修改文件的最后时间

```
1   #通过 touch -d 设置
2   touch -d "2023-10-01 12:34:56" example.txt
3
4   说明: 表示将example.txt文件的最后修改时间为 2023-10-01 12:34:56
```

- 将 3 天前的日志文件复制到 /backup/old_logs 目录
- 删除 5 天前的日志文件

3.2.2 清理临时文件

某临时目录 /tmp/userdata 中存在大量临时文件（文件名包含 tmp_ 前缀），要求：

- 查找所有 24 小时内修改过的 tmp_* 文件并移动到 /tmp/recent
- 删除超过 30 天未修改的 tmp_* 文件

```
1   初始化数据:
2   # 创建 userdata目录
```

```
mkdir -p /tmp/userdata
cd /tmp/userdata
# 在目录下， 创建一些以tmp_开头的文件
touch tmp_01.txt
touch tmp_02.txt
touch tmp_03.txt
touch tmp_04.txt
touch tmp_05.txt
touch tmp_06.txt

# 修改文件的最后修改时间， 以便于后续需求查询验证工作
touch -d '2025-02-01 15:00:00' tmp_01.txt
touch -d '2023-08-15 15:00:00' tmp_03.txt
touch -d '2025-03-01 15:00:00' tmp_05.txt
touch -d '2025-02-25 15:00:00' tmp_06.txt

# 需求一： 查找所有 24 小时内修改过的 tmp_* 文件并移动到 /tmp/recent
# 查找所有 24 小时内修改过的 tmp_* 文件
cd /tmp/userdata
find ./ -mtime -1 -type f -name 'tmp_*'
# 并移动到 /tmp/recent
# 验证： 目标目录是否存在 如果没有先创建
mkdir -p /tmp/recent
# 执行移动操作： mv 要移动的内容 目的地
find ./ -mtime -1 -type f -name 'tmp_*' -exec mv {} /tmp/recent \;
# 验证是否移动成功:
# 步骤一： 查看 /tmp/userdata 下 02 和 04 是否存在 【正常应该是不存在】
# 步骤二： 查看 /tmp/recent 下 02 和 04 是否存在 【正常应该是存在】
```

3.2.3 查找特定文件并归档

某服务器 /usr/lib 目录下可能存在遗留的大文件（超过 100MB），要求：

- 查找所有大小超过 100MB 的文件。

```
# 检验： /usr/lib 是否存在 【存在】
# 需求实现
find /usr/lib -size +100M -type f

#尝试校验:
ll -h 跟上find查询到文件路径，确认是否是大于100M
```

- 将这些文件的路径记录到 /var/log/large_files.list。

```
注意： 目前， find目录执行完后， 直接将查询的结果， 输出到控制台， 需求要求直接输出到一个文件中， 此时相当于要改变输出的方式

此时， 一般要使用的就是一个新的命令， 重定向符号
> 和 >> ： 重定向符号
    作用： 将执行的结果， 通过重定向符号， 直接输出到另一个地方
        > 覆盖
        >> 追加
    使用格式： 命令1 > 目的地

    注意： 输出的目的地文件如果不存在， 重定向会自己创建
写法：
find /usr/lib -size +100M -type f > /var/log/large_files.list

表示将查询的结果， 通过重定向的符号，覆盖到 /var/log/large_files.list文件中
```

- 将这些文件复制到 /archive/large_files 目录

```
# 验证： 复制的目的地是否存在，如果不存在， 需要先创建
mkdir -p /archive/large_files
# 执行
find /usr/lib -size +100M -type f -exec cp {} /archive/large_files \;
#校验： 直接进入到 /archive/large_files 查看是否有大于100M文件复制过来
cd /archive/large_files/
ll -h
```

3.3 编辑文件 vi、vim

vi是visual interface的简称, 是Linux中最经典的文本编辑器（Windows中的记事本）

vi的核心设计思想：让程序员的手指始终保持在键盘的核心区域, 就能完成所有编辑操作

vi的特点：

- 只能是编辑文本内容, 不能对字体段落进行排版
- 不支持鼠标操作
- 没有菜单
- 只有命令

vim 是从vi发展出来的文本编辑器, 支持代码补全、编译及显示效果等方面编程的功能提别丰富, 在程序员中被广泛使用, 被称为编辑器之神。

在CentOS Stream 9系统中，我们可以通过dnf命令安装vim编辑器：

```
dnf -y install vim
```

3.3.1 打开文件

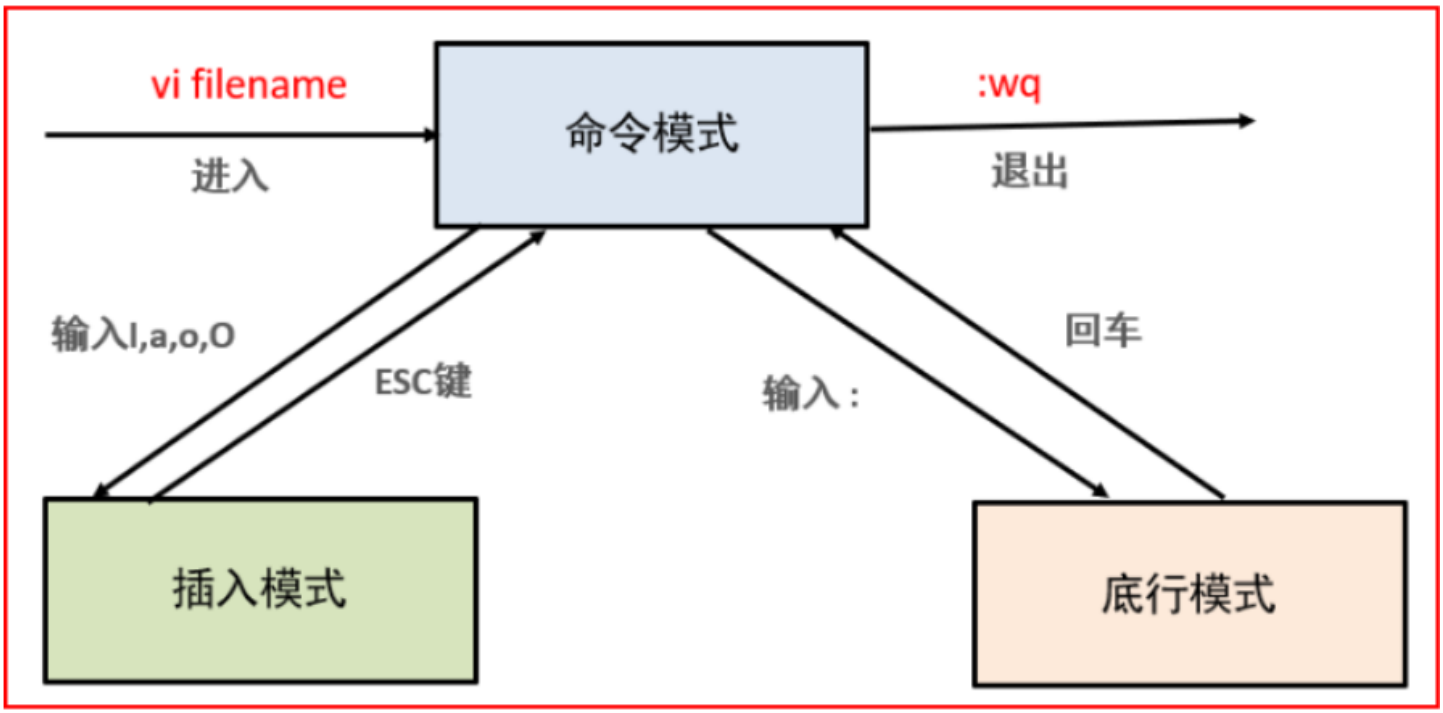
```
vi a.txt          # 直接打开文件
vim a.txt         # vim是vi的增强版
vim +10 a.txt     # 直接打开文件，并定位到第10行
```

vi/vim三种工作模式： 命令模式、插入模式(编辑模式)、末行模式(底行模式)

命令模式：复制、粘贴、移动光标、撤销与恢复

插入模式（编辑模式）：只能编辑文件内容（写字）

底行模式（末行模式）：保存文件、退出文件、显示行号、搜索关键词



3.3.2 命令模式

当我们通过vi/vim命令打开文件时，默认就处于**命令模式**

> 小技巧：进入vim编辑器，先查看左下角有没有提示信息，如果没有任何信息，代表你当前位于命令模式！

命令	功能
o	在当前行后面插入一空行
O	在当前行前面插入一空行

dd	剪切或删除光标所在行
ndd	从光标位置向下连续剪切或删除 n 行
yy	复制光标所在行
nyy	从光标位置向下连续复制n行
p	粘贴
u	撤销上一次命令，相当于Windows中的Ctrl + Z
gg	回到文件顶部
G	回到文件末尾
Ctrl + R	恢复，与u相对应

3.3.3 编辑模式

如何进入编辑模式呢?

答:

- `i`：**在当前光标的前面插入内容**
- `a`：在当前光标的后面插入内容
- `o`：在光标的后一行插入内容
- `O`：在光标的前一行插入内容

问题：如何从编辑模式回到命令模式

答：按 `Esc` 键

3.3.4 底行模式

核心重点记住3个字母即可，:w、:q、！ :x

在Linux操作系统中，文件必须先保存后退出!

！叹号代表强制，强制保存、强制退出、强制保存并退出!

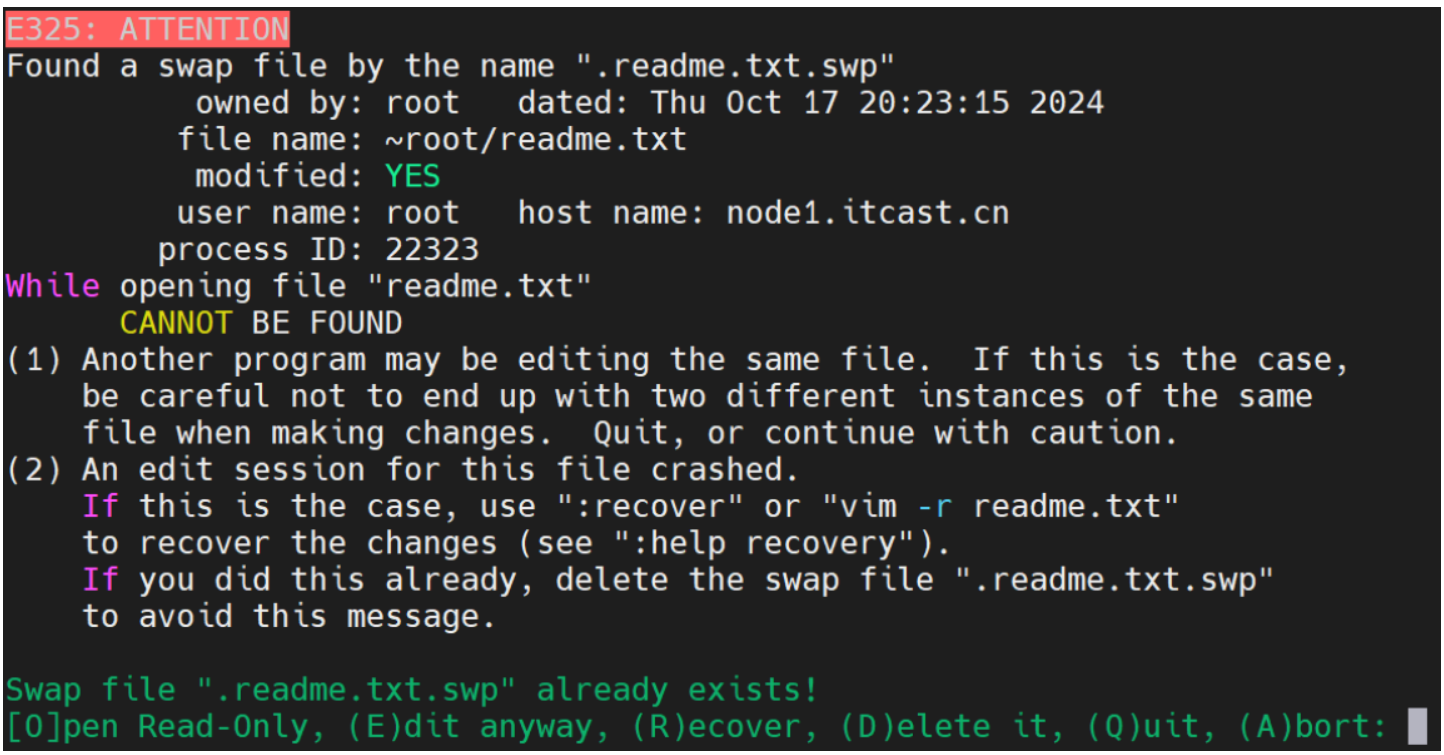
命令	**功能**
:w 文件	另存为
:w	保存(ctrl + s)
:q	退出, 如果没有保存,不允许退出

:q!	强行退出, 不保存退出
:wq	保存并退出
:x	保存并退出 【建议运维选择】
:set nu	设置行号，取消行号使用:set nonu
:%s/旧关键词/新关键词/g	文本替换
:noh	取消高亮
/关键词	搜索某一关键词

:wq和:x区别?

> 答：如果文件内容有改变，两者的效果是一样的。如果文件内容没有改变，:x不会改变文件的最后修改时间，但是:wq会更新文件的最后修改时间。
>
> **运维工程师,强烈推荐使用:x**

3.3.5 编辑常见问题



如果在打开某个文件时，弹出以上提示，那代表你这个文件之前没有保存就强制退出了，触发了Vim的备份机制，产生了一个`.文件名称.swp`交换文件。

以后每次打开之前的文件就会产生上面的提示，解决方案：

① 之前的修改不重要，可以直接删除的情况

- 直接回车，切换到错误的底部，然后按q，直接退出，然后执行删除操作

```
rm  .源文件名称.swp
```

② 之前的修改很重要，需要先恢复内容，然后再解决报错问题

- 第一步：直接回车，切换到错误的底部，找到回复菜单，一般是R，恢复文件内容
- 第二步：针对找回的内容进行:x保存并退出
- 第三步：删除刚才产生的交换文件

```
rm  .源文件名称.swp
```

3.4 文本查看命令

- 1- cat命令【重要】
  - 作用：从上往下查看文件内容，一次性加载所有数据，适合小文件查看

```
格式:   cat 文件

案例说明:
需求:  查看家目录中anaconda-ks.cfg文件
cat anaconda-ks.cfg
```

- 2- more命令
  - 用于分页查看文件内容的命令

```
语法:
    more  filename

进入文件后，可使用的相关操作:

空格键：向下滚动一页。
Enter键：向下滚动一行。
b：向上翻页（可能不支持所有系统）。
q：退出查看

案例:
    more anaconda-ks.cfg
```

- 3- less命令【重要】

- `less` 命令与 `more` 类似，但功能更强大，它允许用户**向前和向后滚动文件**，同时提供了更多的操作选项

```
语法： less  filename

进入文件后，可使用的相关操作：

空格键：向下翻页。
b：向上翻页。
Page Up/Page Down：向上/向下滚动。 【通过方向键 上下来完成滚动】
q：退出查看。
/搜索词：搜索某个关键词，并跳转到该词的位置。 【可以检索操作】
n：跳到下一个匹配的搜索结果。
h：查看帮助信息

案例演示：
        less anaconda-ks.cfg
```

【退出操作】

```
强制退出： ctrl + c （大多数）  ctrl + d （少数）

其他退出方式：q  quit exit
```

- 4- head命令【了解】
  - 作用：查看文件的前N行内容，默认显示前10行
  - 格式：head -N 文件

```
N：  表示从前往后看N行

案例说明：
需求： 查看家目录中anaconda-ks.cfg文件的前10行
head anaconda-ks.cfg

需求： 查看家目录中anaconda-ks.cfg文件的前20行
head -20 anaconda-ks.cfg
```

- 5- tail命令【重要】
  - 作用： 用于从后往前看文件，并支持监控文件变化

- 语法：tail -[N][f] 文件

```
N:  表示从后往前看N行
f:  是否持续显示文件末尾内容

例如:
    tail -100 xx.log  查看某个文件最后100行的内容
    tail -f xx.log  实施查看该文件的输出内容
    tail -100f xx.log  默认显示文件末尾100行的内容，并实时监测最新的数据

后期主要使用这个命令。 来查看系统运行的日志


额外学习了一个小命令：echo   打印输出命令， 用于进行输出到控制台
    例如:  输出 111
        echo '111'
```

- 6- wc命令
  - 作用: 用于统计文件或输入中的**行数**、**单词数**和**字节数**
  - 语法: wc [选项] [文件]

选项	说明
-l	统计行数（lines）
-w	统计单词数（words）
-c	统计字节数（bytes）
-m	统计字符数（characters）
-L	统计最长行的长度（length）

- 7- grep命令
  - 作用: 在文件中直接找到包含指定信息的那些行，并把这些信息显示出来
  - 语法: grep 要查找的内容 文件名

```
示例:
    grep network boot.log
    含义：在boot.log文件中，查找包含network的行

```

```
    grep network /var/log/*
    含义：在/var/log目录下的所有文件中，查找包含network的行


扩充点： |
如何打印|：  shift + 反斜杠按键    （英文模式下）

| 表示管道符号
    格式： 命令1 | 命令2 | 命令3 ....
    作用： 将上一个命令的执行结果， 作为下一个命令的输入

    例如：
        cat a.txt b.txt | grep linux

        对cat执行结果，通过grep查询包含linux的行内容

        并不是所有的命令都支持管道符的
```

3.5 文件查看案例练习

3.5.1 日志文件分析

- 需求1: 使用 cat 查看 /var/log/messages 的内容。
- 需求2: 使用 head 查看该日志文件的前20行。
- 需求3: 使用 tail 查看该日志文件的最新50行。
- 需求4: 使用 less 浏览该日志文件，查找 "error" 关键字。
- 需求5: 使用 wc 统计 /var/log/syslog 文件的行数、单词数和字符数。

答案：

```
需求1: 使用 cat 查看 /var/log/messages 的内容。
cat /var/log/messages
需求2: 使用 head 查看该日志文件的前20行。
head -20 /var/log/messages
需求3: 使用 tail 查看该日志文件的最新50行。
tail -50 /var/log/messages
需求4: 使用 less 浏览该日志文件，查找 "error" 关键字。
less /var/log/messages
进入后: 输入 /error  然后通过 n 跳转即可

需求5: 使用 wc 统计 /var/log/messages 文件的行数、单词数和字符数。
wc -l /var/log/messages
```

```
wc -w /var/log/messages
wc -m /var/log/messages
```

3.5.2 查找和统计文件中的特定内容

在 /var/log 目录中找到一个大文件（如 messages）。

- 需求1: 使用 grep 命令查找所有包含 "timeout" 的行。
- 需求2: 使用 wc -l 统计包含 "timeout" 的行数。
- 需求3: 使用 less 浏览包含 "timeout" 的日志，确保能查看到整个内容。

```
需求1: 使用 grep 命令查找所有包含 "timeout" 的行。
cd /var/log/
grep timeout messages
需求2: 使用 wc -l 统计包含 "timeout" 的行数。
grep timeout messages | wc -l
需求3: 使用 less 浏览包含 "timeout" 的日志，确保能查看到整个内容。
玩法一:
    less messages
    通过 /timeout  然后使用 n 跳转查看

玩法二:
    grep timeout messages | less

玩法三:
    less messages  | grep timeout | less
```

3.6 文件解压缩

`tar`（tape archive）是一个用于打包和压缩文件的命令行工具。它可以将多个文件和目录打包成一个文件，也可以对这些文件进行压缩，减少存储空间。`tar` 命令广泛用于 Unix 和类 Unix 系统，包括 Linux 和 macOS。

- tar命令:
  - 格式: tar [选项] 文件 ...
  - 选项说明【了解】:

选项	说明
-c	创建一个新的归档文件; 这个选项表示将指定的文件或目录打包成一个新的归档文件。
-x	解压缩归档文件 ; 使用该选项从归档文件中提取文件。
-f	指定归档文件的名称; 该选项必须紧跟着归档文件名，指定要创建或提取的归档文件。
-z	使用 gzip 压缩或解压归档文件; 该选项表示将归档文件压缩成 .tar.gz 格式，或者解压 .tar.gz 格式的归档文件。
-v	在创建或解压归档时显示详细过程信息（列出文件名）; 这个选项会显示每个文件在归档过程中的详细信息，通常用于调试或跟踪进度。
-j	使用 bzip2 压缩或解压归档文件; 该选项表示将归档文件压缩成 .tar.bz2 格式，或者解压 .tar.bz2 格式的归档文件。
-J	-J (xz) 使用 xz 压缩或解压归档文件; 该选项表示将归档文件压缩成 .tar.xz 格式，或者解压 .tar.xz 格式的归档文件。
-C	用于指向新的解压的目录 C是大写的C

- 常用组合【重点】

参数组合	说明
-cvf	创建归档文件,俗称 "打包",打包后的文件后缀名为.tar
-xvf	解压归档文件, 俗称 "拆包"
-czvf	创建归档文件，并采用gzip进行**压缩**, 压缩后的文件后缀名为.tar.gz
-xzvf	**解压**被gzip压缩的归档文件
-cjvf	创建归档文件，并采用bzip2进行**压缩**, 压缩后的文件后缀名为.tar.bz2
-xjvf	**解压**被bzip2压缩的归档文件

```
案例:
    需求1:  请 /root目录下所有内容，全部压缩为一个文件，文件名字: root.tar.gz
```

```
    tar -czvf root.tar.gz 公共/ 模板/ 视频/ 图片/ 文档/ 下载/ 音乐/ 桌面/ access_2025022* a.txt
    或者
    tar -czvf root.tar.gz *      此处*表示当前目录下所有内容


    需求2：将 /root目录下 除了压缩包，其余的都删除后，尝试进行解压操作
    cd /root
    mv root.tar.gz /
    rm -rf /root/*
    尝试解压操作：  将根目录下 root.tar.gz 解压到 /root下
    tar -xzvf root.tar.gz -C /root
```

- zip和unzip命令
  - 作用: zip用于压缩, unzip进行解压 针对是zip压缩包
  - 格式:
    - zip [-r] 归档文件名.zip 需要归档的文件列表

选项	说明
-r	递归压缩（压缩文件夹）

    - unzip 归档文件名.zip [-d 解压目录]

选项	说明
-d	如果不指定 表示解压到当前路径

```
首先：
    在家目录下， 创建 4个文件， 分别为 test01.txt test02.txt test03.txt test04.txt
接着：
    在家目录下， 创建一个 test01目录， 将 test03.txt、test04.txt文件放置到test01目录中

cd ~
touch  test01.txt test02.txt test03.txt test04.txt
mkdir -p test01
mv test03.txt test04.txt test01

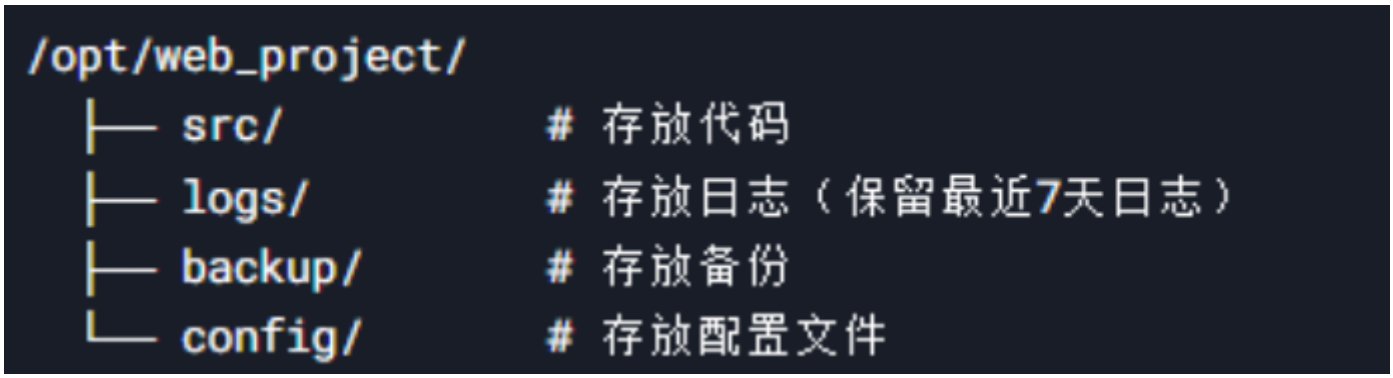
需求一：将test01和test01.txt以及test02.txt进行打包为zip， zip文件名字为test.zip
cd ~
```

```
zip -r test.zip test01/ test01.txt test02.txt
需求二： 将 test.zip 解压到 /root/test02目录中， 如果目录不存在，可以先创建
unzip /root/test.zip -d /root/test02/
```

3.7 综合练习

3.7.1 创建项目目录与日志管理

1- 创建目录结构：



2- 生成模拟日志

- 需求一: 在logs/目录下创建 3个日志文件，命名格式为 access_2023-08-0{1..3}.log
- 需求二: 在每个日志文件中写入 5 行文本，内容为 "[INFO] Request processed at 日期"

3- 备份与清理

- 需求一: 将logs/目录下所有以 2023-08-0 开头的日志文件，复制到backup/目录
- 需求二: 保留logs/目录中仅最近1天的日志（文件名日期最大的3个文件），其余删除 （rm直接删除即可）

```
需求一：
mkdir -p /opt/web_project
cd /opt/web_project
mkdir -p src logs backup config

校验: tree /opt/web_project
结果:
/opt/web_project/
├── backup
├── config
```

```
├── logs
└── src

2- 生成模拟日志
需求一: 在logs/目录下创建 3个日志文件，命名格式为 access_2023-08-0{1..3}.log
cd /opt/web_project/logs
touch access_2023-08-01.log access_2023-08-02.log access_2023-08-03.log

需求二: 在每个日志文件中写入 5 行文本，内容为 "[INFO] Request processed at 日期"
cd /opt/web_project/logs

方式一: 使用 vi/vim
vi access_2023-08-01.log
输入 i 进入插入模式， 写入以下内容
[INFO] Request processed at 2023-08-01 15:20:20
[INFO] Request processed at 2023-08-01 15:20:21
[INFO] Request processed at 2023-08-01 15:20:22
[INFO] Request processed at 2023-08-01 15:20:23
[INFO] Request processed at 2023-08-01 15:20:24

esc 退出插入模式， 输入 :x 保存退出

方式二:  使用echo

echo '[INFO] Request processed at 2023-08-02 18:15:15' >> access_2023-08-02.log
echo '[INFO] Request processed at 2023-08-02 18:15:16' >> access_2023-08-02.log
echo '[INFO] Request processed at 2023-08-02 18:15:17' >> access_2023-08-02.log
echo '[INFO] Request processed at 2023-08-02 18:15:18' >> access_2023-08-02.log
echo '[INFO] Request processed at 2023-08-02 18:15:19' >> access_2023-08-02.log

使用 cat access_2023-08-02.log  校验是否有五条数据


echo '[INFO] Request processed at 2023-08-03 20:20:15' >> access_2023-08-03.log
echo '[INFO] Request processed at 2023-08-03 20:20:16' >> access_2023-08-03.log
echo '[INFO] Request processed at 2023-08-03 20:15:17' >> access_2023-08-03.log
echo '[INFO] Request processed at 2023-08-03 20:20:18' >> access_2023-08-03.log
```

```
echo '[INFO] Request processed at 2023-08-03 20:20:19' >> access_2023-08-03.log

使用 cat access_2023-08-03.log  校验是否有五条数据

3- 备份与清理
需求一: 将logs/目录下所有包含 2023-08-0 文件名的日志文件，复制到backup/目录
find /opt/web_project/logs -name '*2023-08-0*' -type f -exec cp {} /opt/web_project/backup/ \;

校验:
cd /opt/web_project/backup/
ll
查看是否有对应的文件产生

需求二: 保留logs/目录中仅最近1天的日志（文件名日期最大的3个文件），其余删除 （rm直接删除即可）
cd /opt/web_project/logs/
rm -rf access_2023-08-01.log access_2023-08-02.log

```

3.7.2 文件搜索与统计

使用find命令完成以下操作:

- 需求一: 在/etc目录下查找所有后缀为 .conf 的文件，将结果保存到 /opt/config_list.txt
- 需求二: 在用户主目录(~)下查找所有超过 1KB 的 .sh 脚本文件

统计信息:

- 需求一: 统计/opt/config_list.txt文件的总行数（即找到的.conf文件数量）
- 需求二: 查看/var/log/messages的最后20行内容，并统计其中包含 "error" 关键词的行数

```
使用find命令完成以下操作:
需求一: 在/etc目录下查找所有后缀为 .conf 的文件，将结果保存到 /opt/config_list.txt
find /etc -name '*.conf' -type f > /opt/config_list.txt
校验:
cat /opt/config_list.txt

需求二: 在用户主目录(~)下查找所有超过 1KB 的 .sh 脚本文件
cd /root
```

```
find ./ -size +1k -name '*.sh' -type f


统计信息:
需求一: 统计/opt/config_list.txt文件的总行数（即找到的.conf文件数量）
wc -l /opt/config_list.txt

#结果:  292 /opt/config_list.txt
需求二: 查看/var/log/messages的最后20行内容，并统计其中包含 "error" 关键词的行数

tail -20 /var/log/messages | grep error | wc -l

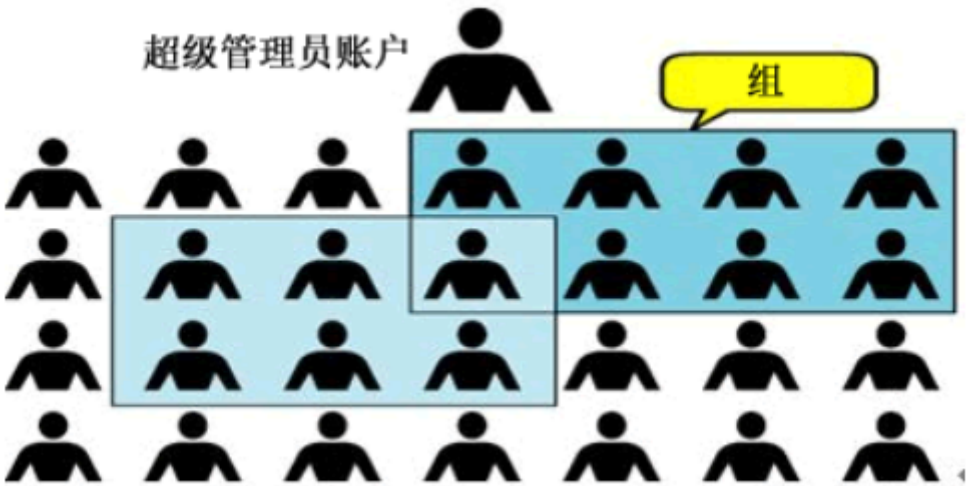
```

4. 用户和组管理

4.1 用户与用户组的基本概念

用户：是系统中的独立身份，用于登录和执行任务。每个用户有唯一的标识符（UID）和自己的文件及设置。用户的主要功能是访问系统资源并运行进程。

用户组：是多个用户的集合，用于简化权限管理。通过将用户加入组，用户可以继承组的权限，从而访问特定的文件、目录或执行操作。



注意：

- 在Linux操作系统中，其实是一个多用户多任务的操作系统。支持多个用户同时登陆Linux操作系统，而且不会相互影响。
- 在Windows操作系统中，其实是一个单用户多任务的操作系统。在同一时间内，只允许登陆一个用户。

4.2 用户组相关操作

在Linux操作系统中，用户组的操作通常有三种情况：用户组的添加、用户组的修改与用户组的删除。

用户组：group

添加：add

修改：mod

删除：del

用户组添加：

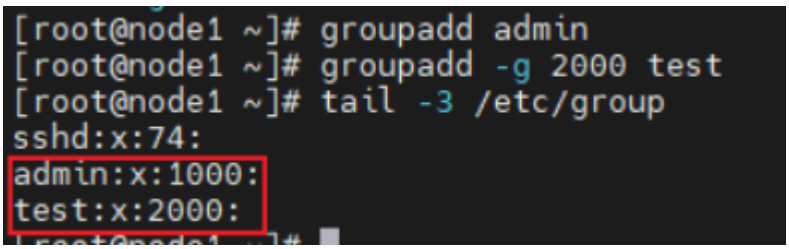
- 语法： groupadd [选项] 用户组名称
- 选项：
  - -g : gid 代表创建用户组时指定用户组的组ID编号。root超级管理员组编号为0，系统组编号1-999，自定义组编号(自己创建的组)从1000开始
- 案例：

```
案例一：在系统中创建一个admin用户组
groupadd admin

案例二：在系统中创建一个test组，组编号为2000
groupadd -g 2000 test

查看是否创建成功，可查询系统的用户组文件：/etc/group

```



该文件说明: 可以发现，文件中，主要分为四列(三个冒号)

- 第一列: 用户组的组名称
- 第二列: 使用x占位，用于存储组密码，真实密码存储在/etc/gshadow文件中(了解)
- 第三列: 用户组的组ID编号
- 第四列: 组内用户信息，简单来说就是这个组是哪个用户的附加组

用户组修改：

- 语法： groupmod [选项] 要修改的用户组名称
- 选项说明：
  - -g: gid编号没代表修改用户组的组ID编号
  - -n: name名称，代表修改用户组的组名称
- 案例：

```
案例一：把test用户组的组ID编号更改为1002
groupmod -g 1002 test
校验：
cat /etc/group
案例二：把admin用户组的组名称更改为admins
groupmod -n admins  admin
校验：
cat /etc/group

案例三：  同时修改组ID和组名字
groupmod -n test01 -g 1003 test
校验：
cat /etc/group
```

```
[root@node1 ~]# groupmod -g 1002 test
[root@node1 ~]# groupmod -n admins admin
[root@node1 ~]# tail -10 /etc/group
polkitd:x:994:
rtkit:x:172:
geoclue:x:993:
pipewire:x:992:
flatpak:x:991:
chrony:x:990:
pesign:x:989:
sshd:x:74:
test:x:1002:
admins:x:1000:
[root@node1 ~]#
```

用户组删除:

- 语法: groupdel 要删除用户组的组名称
- 案例: groupdel test

4.3 用户相关操作

在Linux操作系统中，用户的管理涉及用户的添加、修改和删除。

用户：user

添加：add

修改：mod

删除：del

用户添加

- 基本语法: useradd [选项1 选项1的值 选项2 选项2的值 ...] 用户名称
- 选项:

选项	说明
-g	gid，添加用户时指定用户所属的主组的编号（gid），只能有一个
-G	GID，添加用户时指定用户所属的附加组的编号（GID），可以有多个，可以使用逗号隔开
-u	指定用户的ID编号（必须是唯一的，0超级管理员的编号，1-999系统用户编号，1000以后自定义用户的编号）
-d	添加用户时指定用户的家目录，如果不指定，默认为/home/用户名
-s	指定用户可以使用的Shell脚本，主要有两个/bin/bash（普通用户）、/sbin/nologin（系统用户，不能用于登录操作系统），默认创建的就是普通用户
-r	代表创建系统账号 一般与-s /sbin/nologin 组合使用 表示为系统用户 系统用户： 一般主要是用于启动特定的应用程序

- 案例:

```
案例一：创建一个账户 zhangsan
useradd zhangsan
查看用户：
id zhangsan
id命令获取用户的信息：
用户id=1000(zhangsan) 组id=1003(zhangsan) 组=1003(zhangsan)

注意：默认情况下，如果我们创建用户时不指定用户的所属组，则Linux操作系统会自动帮我们生成一个与用户名同名的用户组，然后将其追加到这个组中

案例二：创建账户时指定用户所属的用户组为admins
# 如何验证是否有这个用户组呢?
```

```
grep admins /etc/group
如何查不到，说明不存在， 创建该组
groupadd admins

#创建用户， 添加到admins 主组
useradd -g admins lisi

案例三：创建一个账户wangwu 其UID编号为2000

useradd -u 2000 wangwu

案例四：在/斜杠根目录下创建一个uhome文件夹，然后创建zhaoliu并指定zhaoliu的家目录为/uhome/zhaoliu
mkdir -p /uhome

useradd -d /uhome/zhaoliu zhaoliu

案例五：创建一个系统账号mysql，禁止其登录Linux操作系统

useradd -r -s /sbin/nologin mysql

选项说明:
-r ：代表创建系统账号

查看与用户相关的文件/etc/passwd
cat /etc/passwd
```

```
[root@node1 ~]# cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
adm:x:3:4:adm:/var/adm:/sbin/nologin
lp:x:4:7:lp:/var/spool/lpd:/sbin/nologin
sync:x:5:0:sync:/sbin:/bin/sync
shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
halt:x:7:0:halt:/sbin:/sbin/halt
mail:x:8:12:mail:/var/spool/mail:/sbin/nologin
operator:x:11:0:operator:/root:/sbin/nologin
games:x:12:100:games:/usr/games:/sbin/nologin
ftp:x:14:50:FTP User:/var/ftp:/sbin/nologin
nobody:x:65534:65534:Kernel Overflow User:/:/sbin/nologin
systemd-coredump:x:999:997:systemd Core Dumper:/:/sbin/nologin
dbus:x:81:81:System message bus:/:/sbin/nologin
tss:x:59:59:Account used for TPM access:/:/usr/sbin/nologin
stapunpriv:x:159:159:systemtap unprivileged user:/var/lib/stapunpriv:/sbin/nologin
sssd:x:998:995:User for sssd:/:/sbin/nologin
polkitd:x:997:994:User for polkitd:/:/sbin/nologin
rtkit:x:172:172:RealtimeKit:/:/sbin/nologin
geoclue:x:996:993:User for geoclue:/var/lib/geoclue:/sbin/nologin
pipewire:x:995:992:PipeWire System Daemon:/run/pipewire:/usr/sbin/nologin
flatpak:x:994:991:User for flatpak system helper:/:/sbin/nologin
chrony:x:993:990:chrony system user:/var/lib/chrony:/sbin/nologin
pesign:x:992:989:Group for the pesign signing daemon:/run/pesign:/sbin/nologin
sshd:x:74:74:Privilege-separated SSH:/usr/share/empty.sshd:/usr/sbin/nologin
zhangsan:x:1000:1003::/home/zhangsan:/bin/bash
lisi:x:1001:1000::/home/lisi:/bin/bash
wangwu:x:2000:2000::/home/wangwu:/bin/bash
zhaoliu:x:2001:2001::/uhome/zhaoliu:/bin/bash
mysql:x:991:988::/home/mysql:/sbin/nologin
```

在Linux操作系统中，一切皆文件，与用户相关的信息也是保存在一个文件中，这个文件就是/etc/passwd，观察以上文件的运行结果可知，/etc/passwd文件一共有6个冒号，共7列

第1列：用户名称
第2列：用户的密码，使用一个x占位符，真实的用户密码/etc/shadow文件中
第3列：用户的uid编号（必须是唯一的）
第4列：用户组的组ID编号（gid）
第5列：用户的备注信息，可以通过useradd -c来进行指定 useradd -c "zhangsan" zs
第6列：用户的家目录（大部分用户的家都在/home目录下，超级管理员的家在/root目录下）
第7列：用户的Shell脚本，/bin/bash普通用户，/sbin/nologin系统用户、账号被禁用

> 扩展：与用户相关的密码文件 `/etc/shadow`
>
> vim /etc/shadow
>
> root:6b/H9yGx2gtTHf8n2$4zJWKJt55ulVbB6exIsH.zSgLPdF06SLj75IeDqtahrhXtnJmRHF4DdtC7iych6qd9l4D1r8xGOULqf.LPFw/1
>
> 第1列：用户名称
>
> 第2列：加密后的用户密码

用户修改

- 语法: usermod [选项 选项的值 ...] 要修改的用户名称
- 选项:

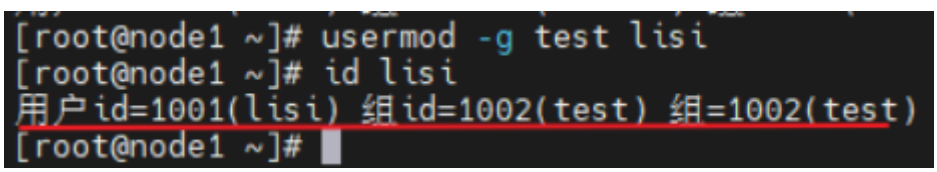
选项	说明
-g	gid，修改用户时指定用户所属的主组的编号（gid），只能有一个
-G	GID，修改用户时指定用户所属的附加组的编号（GID），可以有多个，可以使用逗号隔开 如果需要在原有附加组中增加其他的组, 需要使用 -a 参数
-u	指定用户的ID编号（必须是唯一的，0超级管理员的编号，1-999系统用户编号，1000以后自定义用户的编号）
-d	修改用户时指定用户的家目录，要结合-m一起使用:搬家时把家具一起搬过去
-s	指定用户可以使用的Shell脚本

- 扩展选项：

选项	说明
-L	Lock，锁定用户，一个用户如果被锁定，则这个用户无法登陆Linux系统
-U	Unlock，解锁用户

```
案例一：修改lisi账号的所属的主组为test
# 如何验证是否存在test组呢?
grep test /etc/group
# 如何没有任何的显示， 说明不存在， 执行创建
groupadd test

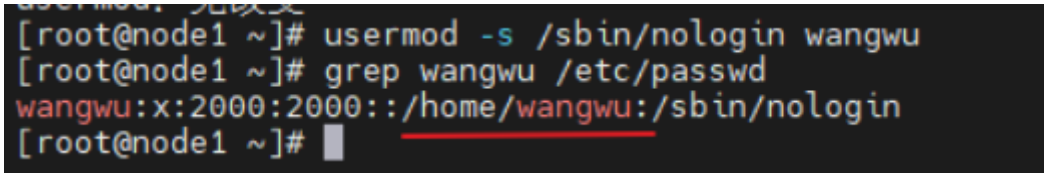
# 修改用户主组
usermod -g test lisi
```




```
案例二：修改用户的家目录，把zhaoliu的家目录更改为/home/zhaoliu
usermod -d /home/zhaoliu -m zhaoliu

```

```
4   案例三：禁止wangwu账号登录Linux操作系统
5   usermod -s /sbin/nologin wangwu
```



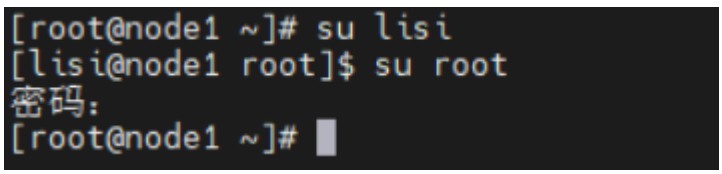
- 设置密码: passwd
  - 语法: passwd 要修改密码的用户名

```
1   案例：修改zhangsan账号的密码为123456
2   passwd zhangsan
```

> 说明：超级管理员给某个用户重置密码，只需要直接输入两次密码即可（支持弱密码），但是普通用户重置密码，首先要输入当前的密码，然后还需要输入两次新密码，而且这个密码有要求：有大写、有小写、有数字且长度需要大于8位。

- 切换登录用户命令: su
  - 语法: su [-] 要切换的用户名称

> 注意：
> a. 从root 往普通用户切换不需要密码，但是反之则需要root 密码；
> b. 切换用户之后前后的工作路径是不变的，添加了选项[-]会自动切换到用户的家；
> c. 普通用户没有办法访问root 用户家目录，但是反之则可以；



- 用户的删除操作
  - 语法: userdel [选项] 要删除用户的名称
  - 选项:
    - -r： 删除用户的同时删除用户的家目录
    - -f： 主要用于删除正在登陆的用户，相当于强制删除（特殊情况下才需要使用）

```
1   案例：删除wangwu这个账号
2   userdel wangwu
```

```
3
4  案例：删除lisi的同时删除用户的家
5  userdel -r lisi
6
7  案例：强制删除系统中登陆的某个账号
8  userdel -rf zhaoliu
```

4.4 综合案例

假设你是一家公司的系统管理员，需要为新项目组设置用户和用户组。项目组名为 `project_team`，组内有三个成员：`alice`、`bob` 和 `charlie`。你需要完成以下任务：

1. 创建一个名为 `project_team` 的用户组。

```
1  groupadd project_team
2
3  校验:
4  grep project_team /etc/group
```

2. 创建三个用户：`alice`、`bob` 和 `charlie`，并将他们的默认 Shell 设置为 `/bin/bash`。

```
1  useradd -s /bin/bash alice
2  useradd -s /bin/bash bob
3  useradd -s /bin/bash charlie
4
5
6  校验:
7  tail -10 /etc/passwd
```

3. 将这三个用户加入 `project_team` 用户组。

```
1  usermod -G project_team  alice
2  usermod -G project_team  bob
3  usermod -G project_team  charlie
4
5  校验:
6  grep project_team  /etc/group
7
8  结果:
9  project_team:x:1001:alice,bob,charlie
```

4. 检查用户和用户组是否配置正确。
5. 删除 `charlie` 用户，并将其从 `project_team` 组中移除。

```
userdel -r charlie

# 校验
grep project_team  /etc/group
project_team:x:1001:alice,bob
```

5. 权限与所有权管理

5.1 权限管理

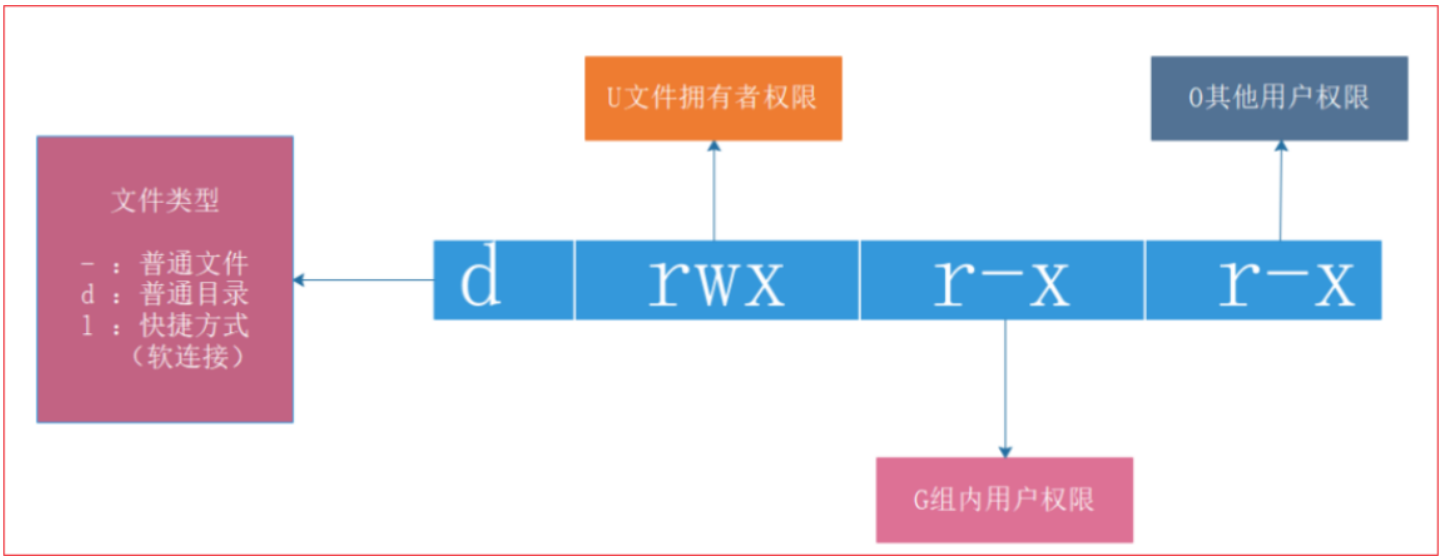
在多用户计算机系统的管理中，权限是指**某个特定的用户具有特定的系统资源使用权利**。

在Linux操作系统中，权限只有三种情况：可读、可写、可执行

	权限针对文件	权限针对目录
读 r	表示可以查看文件内容；cat	表示可以(ls)查看目录中存在的文件名称
写 w	表示可以更改文件的内容；vim 修改，保存退出	表示是否可以删除目录中的子文件或者新建子目录(rm/touch/mkdir)
执行 x	表示是否可以开启文件当中记录的程序,一般指二进制文件(.sh)	表示是否可以进入目录中(cd)



5.1.1 权限的详解




```
第一位: 表示文件类型
    - : 普通文件
    d : 普通目录
    l : 快捷方式(软连接)

第二位 ~ 第四位: 表示所属用户权限(代号:U)

第五位 ~ 第七位: 表示所属用户组权限(代号:G)

第八位 ~ 第十位: 表示其他用户权限(代号:O)

特殊用户: root，在Linux操作系统中，root用户时一个特殊的存在，其不受权限的影响。其在整个系统中，拥有所有文件的管理权限。
```

5.1.2 设置文件或目录的权限

命令:

chmod = change modify

5.1.2.1 字母设置方式

语法: chmod [选项] [权限] 文件名称

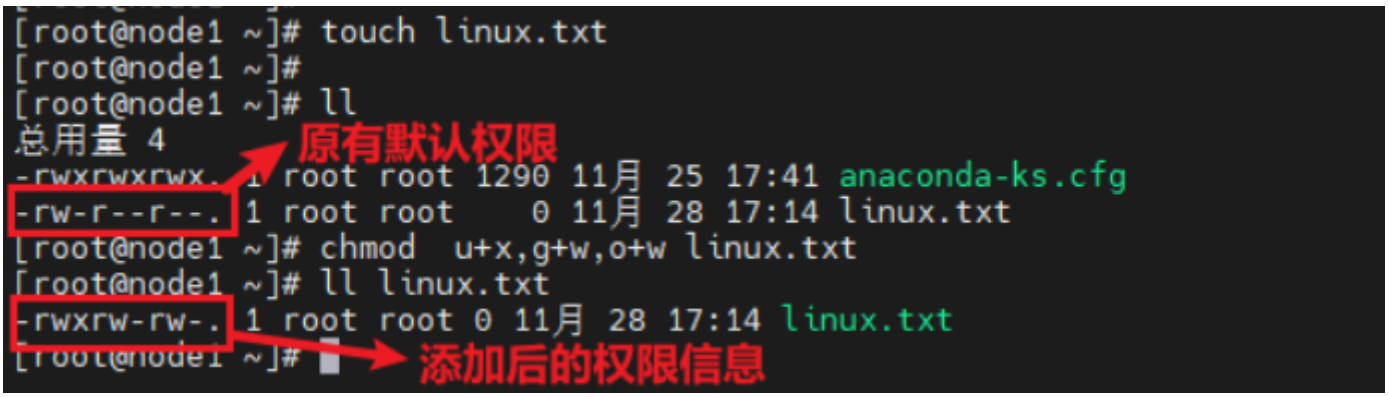
命令	选项	u、g、o或ugo或a	权限设置	权限列表
		u	+ (在原有权限上**增加**权限)	r
chmod	[-R]	g	- (在原有权限上**减少**权限)	w
		o	= (赋权限)	x

```
-R ： 表示递归设置， 当针对目录及其子目录或者子文件进行设置的时候,需要添加
```

案例实施:

为家目录下的创建一个linux.txt文件, 并为此文件拥有者增加一个x权限，组内用户增加一个w权限，其他用户也增加一个w权限

```
chmod u+x,g+w,o+w  linux.txt
或者
chmod u+x,go+w  linux.txt
或者
chmod u=rwx,go=rw  linux.txt
```




```
案例：把家目录下shop文件夹连同其内部的文件中的文件拥有者、所属组内用户、其他用户都设置为rwx（全部权限）
mkdir -p /root/shop
cd  /root/shop
touch a.txt b.txt c.txt

设置权限:
chmod -R ugo=rwx  shop
```

```
案例：减少shop文件夹的权限（连同内部的文件），其文件拥有者权限保持不变，组内用户减少w权限，其他用户减少rwx权限。

chmod -R g-w,o-rwx  shop
```

5.1.2.2 数字设置

文件**权限与数字**的对应关系：

权限	对应数字	意义
r	4	可读
w	2	可写
x	1	可执行

我们在数字形式权限的设置过程中，通常是三位连续的数字，如755、644、777等等

第1位数字代表文件拥有者权限

第2位数字代表文件所属组内用户权限

第3位数字代表其他用户权限

```
# 案例：更改linux.txt文件的权限，文件拥有者拥有全部权限、组内用户拥有可读可写权限、其他用户拥有可读权限

chmod 764 linux.txt

# 案例：给shop文件夹（仅设置文件夹本身），文件拥有者、组内用户、其他用户设置最高权限
chmod 777 shop

# 案例：给文件linux.txt设置权限为600
chmod 600 linux.txt
```

5.1.3 umask值

umask表示创建文件时的默认权限，（即创建文件时不需要设置而天生的权限）

- 查看当前账号的umask值

```
# umask

022
```

那这些umask值和我们的默认权限有何关系？

答：所有文件的默认权限都是由这个文件或目录的最大权限 - umask值

文件的默认最大权限是：666 - 022 = 644 (所属用户可读可写、所属组和其他用户为可读)

目录的默认最大权限是：777 - 022 = 755 (所属用户可读可写可执行、所属组和其他用户为可读可执行)

> **说明**：新创建的文件默认没有执行权限，因此最大权限是 `666`，而目录需要执行权限（进入目录时需要可执行权限），所以其最大权限是 `777`。

5.1.4 【加强】特殊权限

5.1.4.1 冒险位S（set uid）

作用：为了让一般使用者临时具有该文件所属主/组的执行权限。

> 冒险位S，主要针对二进制的脚本文件（命令文件）。查询一个命令所在位置？which

例如：

/usr/bin/passwd在执行它的时候需要去修改/etc/passwd和/etc/shadow等文件，这些文件除了root外，其他用户都没有写权限，但是又为了能让普通用户修改自己的密码，该怎么办呢？

```
ll /usr/bin/passwd
# -rwsr-xr-x. 1 root root 27832 6月  10 2014 /usr/bin/passwd
观察以上命令的运行结果，passwd命令，其文件拥有者权限为rws，这个s就是冒险位权限。

ll /etc/shadow
# ----------. 1 root root 1510 7月   7 16:18 /etc/shadow
```

问题：root超级管理员，可以更改自己的密码，也可以更改别人的密码。但是观察以上图解可知，/etc/shadow文件只能由root用户进行修改与设置。其他用户无权限，那这个时候，冒险位权限就发生作用了。

答：受到冒险位权限的影响。passwd命令，拥有一个s冒险位权限，所以其在运行时，会临时具有root账号的相关功能。

设置冒险位权限：

```
chmod u+s /usr/bin/passwd
# 或
chmod 4755 /usr/bin/passwd
```

取消冒险位权限：

```
chmod u-s /usr/bin/passwd
```

5.1.4.2 粘滞位T（sticky bit）

其主要应用对象为文件夹（目录），这个目录大部分都是公共目录，权限大部分为777。

粘滞位权限：其主要作用就是限制公共目录，让这个目录下的文件不能被误删除。除了这个文件的拥有者，其他用户一律不能删除别人的文件。

- 使用ll -d命令查询/tmp这个公共目录

```
ll -d /tmp
# drwxrwxrwt. 20 root root 4096 7月   9 15:57 /tmp
观察以上命令的运行结果，其其他拥有者权限为rwt，这个t就是粘滞位权限
```

- 删除/tmp目录的粘滞位权限

```
chmod o-t /tmp
```

- 给/tmp目录增加一个粘滞位权限

```
chmod o+t /tmp
# 或
chmod 1777 /tmp
```

5.2 文件所属用户与所属组



5.2.1 所属用户修改

命令：chown = change owner

格式: chown [选项] 新的文件拥有者 文件名称

选项	说明
-R	递归设置，递归更改文件的拥有者

案例：把/root/linux.txt其文件的拥有者更改为itcast

```
1   chown itcast /root/linux.txt
```

案例：把shop文件夹及其内部的文件其文件的拥有者也都更改为itcast

```
1   chown -R itcast  /root/shop
```

5.2.2 所属组修改【了解】

命令：chgrp = change group

格式: chgrp [选项] 新的文件的所属组 文件名称

选项	说明
-R	递归设置，递归更改文件的所属组

案例：把/root/linux.txt其文件的所属组更改为itcast

```
1   chgrp itcast /root/linux.txt
```

案例：把shop文件夹及其内部的文件其文件的所属组也都更改为itcast

```
1   chgrp -R itcast  /root/shop
```

5.2.3 同时修改所属用户和所属组

命令：chown

格式:

```
chown [-R] 文件拥有者:文件的所属组 文件名称
# 或
chown [-R] 文件拥有者.文件所属组 文件名称
```

案例：把/root/linux.txt的文件拥有者和所属组都修改为root

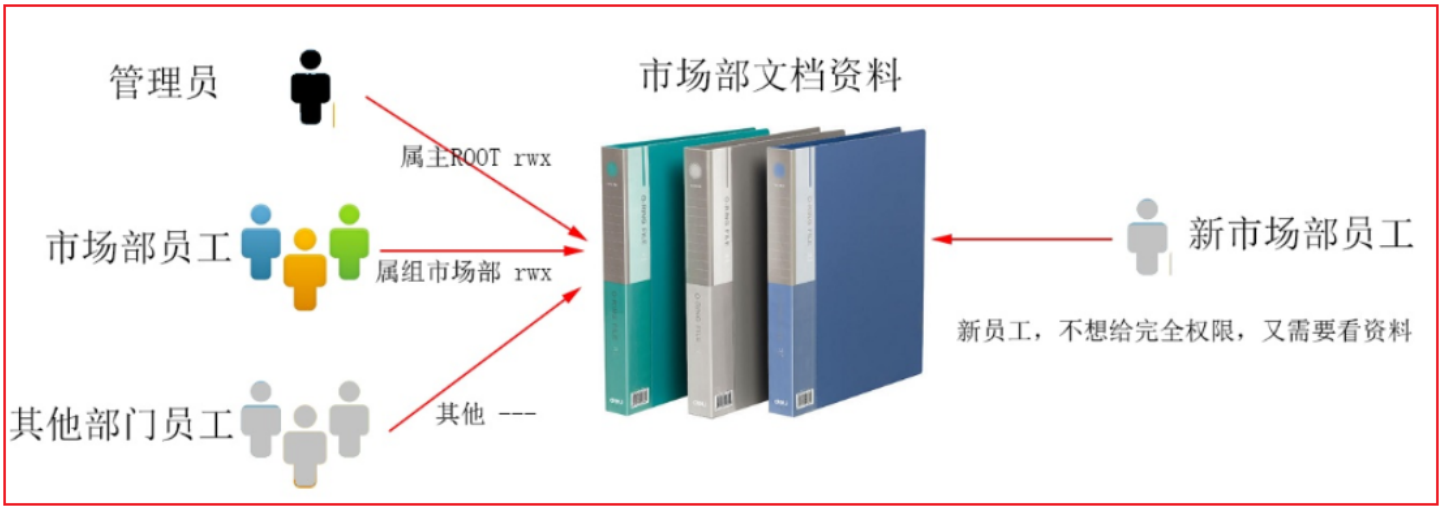
```
chown root:root /root/linux.txt
```

案例：把shop文件夹及其内部的文件其文件的所属组也都更改为root

```
chown -R root:root  /root/shop
```

5.3 ACL权限控制

5.3.1 什么是ACL



ACL，是 Access Control List（访问控制列表）的缩写，在 Linux 系统中， ACL 可实现对**单一用户**设定访问文件的权限。

5.3.2 获取文件的ACL权限

基本语法：getfacl 文件或目录名称

> getfacl = get file acl

案例：获取/root/linux.txt的ACL权限

```
getfacl /root/linux.txt
```

5.3.3 设置文件的ACL权限

基本语法：

```
setfacl [-R] -m u:用户名称:具体权限如rwx 文件或目录名称
# 或
setfacl [-R] -m g:用户组的名称:具体权限如rwx 文件或目录名称


-m: 表示修改（modify）现有的 ACL 规则。
-R: 如需要对目录设置,需要添加-R参数
```

案例：给linux.txt文件添加ACL权限（只针对itcast这个账号）

```
[root@node1-itcast-cn ~]# getfacl linux.txt
# file: linux.txt
# owner: root
# group: root
user::rw-
group::---
other::---

[root@node1-itcast-cn ~]# setfacl -m u:itcast:rwx   linux.txt
[root@node1-itcast-cn ~]# getfacl linux.txt
```

```
# file: linux.txt
# owner: root
# group: root
user::rw-
user:itcast:rwx  # 看到这个信息， 表示已经设置好了ACL权限
group::---
mask::rwx
other::---

```

案例：给shop添加权限，包含shop的子目录和文件（只针对itcast这个用户组）

```
[root@node1-itcast-cn ~]# getfacl shop/
# file: shop/
# owner: root
# group: root
user::rwx
group::rwx
other::rwx

[root@node1-itcast-cn ~]# setfacl -R -m g:itcast:rwx  shop/
[root@node1-itcast-cn ~]# getfacl shop/
# file: shop/
# owner: root
# group: root
user::rwx
group::rwx
group:itcast:rwx
mask::rwx
other::rwx

[root@node1-itcast-cn ~]# cd shop/
[root@node1-itcast-cn shop]# ll
总用量 0
-rwxrwx---+ 1 root root 0  3月  5 15:44 a.txt
-rwxrwx---+ 1 root root 0  3月  5 15:44 b.txt
-rwxrwx---+ 1 root root 0  3月  5 15:44 c.txt
-rwxrwx---+ 1 root root 0  3月  5 15:44 d.txt
[root@node1-itcast-cn shop]# getfacl a.txt
# file: a.txt
# owner: root
# group: root
user::rwx
group::r-x
```

```
group:itcast:rwx
mask::rwx
other::---
```

5.3.4 删除文件的ACL权限

基本语法:

```
setfacl [-R] -x u:用户名称 文件或目录名称
# 或
setfacl [-R] -x g:用户组名称 文件或目录名称


setfacl [-R] -b 文件或目录的名称          =>        删除文件或目录的所有ACL权限
```

案例：删除linux.txt文档的itcast这个用户的ACL

```
setfacl -x u:itcast /root/linux.txt
```

案例：删除shop文件夹的所有ACL权限

```
setfacl -b /root/shop
```

5.4 权限综合案例

5.4.1 权限管理与用户组协作

公司有一个项目组，组内有三个成员：`user1`、`user2`、`user3`。他们需要共同管理一个项目目录`/project`。要求：

1. `/project`目录的所有者为`root`，所属组为`project_group`。
2. `project_group`组的成员可以对`/project`目录下的文件进行读写操作。
3. 其他用户不能访问`/project`目录

5.4.2 ACL权限控制

在 `/shared` 目录下，有一个文件 `report.txt` ，要求：

1. `user1` 可以读写该文件。
2. `user2` 只能读取该文件。
3. 其他用户不能访问该文件。

5.4.3 复杂权限管理

公司有一个 `/data` 目录，目录结构如下：

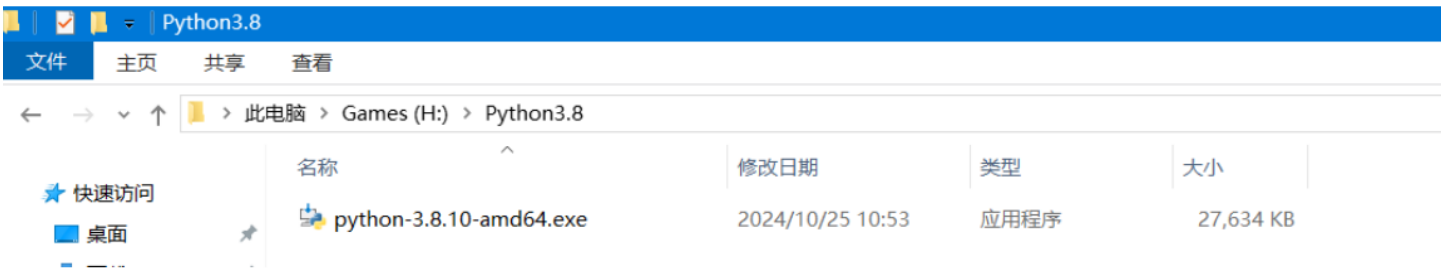
```
/data
├── finance
├── hr
└── it
```

要求：

1. `finance` 目录只能由 `finance_group` 组的成员访问，且只能读写文件，不能删除其他用户的文件。
2. `hr` 目录只能由 `hr_group` 组的成员访问，且只能读取文件。
3. `it` 目录只能由 `it_group` 组的成员访问，且可以读写和执行文件。

6. 软件包管理

6.1 软件包介绍



这个就是Windows系统下的软件安装包（Python软件的安装程序）。Linux下也有很多可以安装的软件，而这些软件的安装包可细分为两种，分别是**源码包**和**二进制包**。

- 源码包：

```
1    软件都是由开发工程师（程序猿）开发而来的。
```

```
软件 => 开发阶段 => 源码（.c/.java）=> 编译过程（gcc => 开发工具） => (.exe/.msi)

Linux系统信奉“开源”，开放源代码。

nginx.tar.gz  =>  源码包

安装比较复杂，需要经过配置、编译、安装过程。
```

- 二进制包：

```
软件 => 开发阶段 => 源码（.c/.java）=> 编译过程（gcc => 开发工具） => (.exe/.msi)
二进制软件包（Linux中的软件包格式为.rpm）

不同的操作系统，二进制包的类型会有所不同。CentOS系列 => *.rpm，Debian => *.deb
```

6.2 包管理工具

在 CentOS Stream 9 中，`dnf` 和 `rpm` 是协同工作的包管理工具，但它们的功能和定位有所不同：

- **`dnf`** 是高级包管理工具，负责解决依赖关系、从仓库下载软件包并进行安装、更新或删除操作。它是用户管理软件包的主要接口。
- **`rpm`** 是底层工具，用于直接处理 RPM 软件包文件（安装、查询、卸载等），不处理依赖关系。它更多地用于系统维护或特定情况下的手动操作。

在Ubuntu Server操作系统中, dpkg 和 apt 是协同工作的包管理工具, dpkg类似于centos stream 9的rpm包管理工具, apt则类似于centos stream 9的dnf包管理工具

6.2.1 rpm包管理操作

格式：rpm [选项]

选项	说明
-a	查询所有套件
-c	只列出组态配置文件，本参数需配合"-l"参数使用
-d	只列出文本文件，本参数需配合"-l"参数使用

-e<套件档>或--erase<套件档>	删除指定的套件
-f<文件>+	查询拥有指定文件的套件
-h或--hash	套件安装时列出标记
-i	显示套件的相关信息
-i<套件档>或--install<套件档>	安装指定的套件档
-l	显示套件的文件列表
-p<套件档>+	查询指定的RPM套件档
-q	使用询问模式，当遇到任何问题时，rpm指令会先询问用户
-R	显示套件的关联性信息
-s	显示文件状态，本参数需配合"-l"参数使用
-U<套件档>或--upgrade<套件档>	升级指定的套件档
-v	显示指令执行过程
-vv	详细显示指令执行过程，便于排错
--force	强制安装
--nodeps	强制卸载，不考虑软件的依赖关系

常用组合说明：

```
# 安装 【重点】
$ rpm -ivh your-package.rpm

# 强制安装 【重点】
$ rpm --force -ivh your-package.rpm

# 卸载
$ rpm -e proftpd-1.2.8
$ rpm -e proftpd-1.2.8 --nodeps    【重点】
--nodeps：强制卸载，不考虑软件的依赖关系

# 列出所有安装过的包
$ rpm -qa
$ rpm -qa | grep sql 【重点】

```

```
# rpm包中的文件安装到那里
$ rpm -ql ***.rpm


# 某个程序是哪个软件包安装
$ rpm -qf `which 程序名`    # 返回软件包的全名
$ rpm -qif `which 程序名`   # 返回软件包的有关信息
$ rpm -qlf `which 程序名`   # 返回软件包的文件列表
```

案例:

```
需求一：尝试通过rpm包安装wget命令：  在今日资料中，提供了一个wget命令的rpm包
rpm -ivh wget-1.21.1-8.el9.x86_64.rpm
或
rpm --force -ivh wget-1.21.1-8.el9.x86_64.rpm

需求二：查看所有安装过的包
操作：rpm -qa

需求三：从所有安装过的包中筛选出wget包
rpm -qa | grep wget

需求四：卸载wget命令
rpm -e wget --nodeps
```

6.2.2 dnf包管理操作

6.2.2.1 dnf的仓库说明

仓库（Repository）是存放软件包的地方，`dnf` 使用仓库来管理软件的安装、更新和删除。

仓库包含多个软件包，并可以有不同的源，例如官方仓库、第三方仓库等。

为什么需要仓库?

- 提供软件包的集中管理。
- 处理软件包的依赖关系，确保系统的一致性。

仓库的分类:

- 默认仓库: CentOS 配置了多个默认仓库： `BaseOS` 、 `AppStream` 和 `Extras`
  - `dnf` 自动使用这些仓库来查找和安装软件包。
  - 如何查看当前系统配置的仓库: dnf repolist
- 添加和管理第三方仓库: 有时需要安装 `dnf` 默认仓库中没有的软件包，可以通过添加第三方仓库来解决
  - 如何添加呢?

```
以 EPEL 仓库为例，如何添加第三方仓库
dnf install epel-release
```

  - 查看所有仓库启用状态

```
dnf repolist all
```

  - 禁用或启用仓库

```
禁用仓库:
dnf config-manager --set-disabled repo_name

启用仓库:
dnf config-manager --set-enabled repo_name
```

说明: 除此之外,还支持配置本地仓库,通过dnf从本地仓库下载

6.2.2.2 dnf命令相关操作

格式： dnf [选项]

类型	选项	说明
安装/更新/删除相关	**install <package>**	安装指定软件包及其依赖
	update <package>	更新指定软件包（不写软件包则更新所有可更新的软件包）
	remove <package>	卸载指定软件包及其依赖
	reinstall <package>	重新安装已安装的软件包

查询相关	list <package>	列出已安装或可用的软件包
	info <package>	显示软件包的详细信息
	search <keyword>	根据关键词搜索可用的软件包
	provides <file>	查找提供某文件的包
系统操作 可以清理安装包以及对软件可以进行更新	check-update	检查系统可用更新
	updateinfo	查看安全更新和其他更新信息
	autoremove	清除未使用的依赖包
	clean all	清理缓存（如元数据、临时文件等）
存储库相关	repolist	列出已启用的仓库
	repoquery <package>	查询仓库中某软件包的信息
调试和高级功能	history	查看或回滚历史操作
	downgrade <package>	降级到软件包的低版本
	group list	列出软件包组
	makecache	生成本地缓存

```
其他选项:
    -y:  在安装过程中,不询问,直接安装

最为核心记忆的单词:
    install remove list


基本语法
# 安装 `httpd`（Apache）服务
dnf -y install httpd

# 更新所有软件包
dnf update

# 删除软件包
dnf remove <package>

# 查看可用更新
dnf check-update

```

```
# 搜索包含关键字的软件包
dnf search <keyword>

# 清理缓存
dnf clean all

# 实用组合
# 更新系统所有包并清理缓存：
dnf update && dnf clean all

# 查询某文件属于哪个软件包：
dnf provides /usr/bin/vim
```

案例实操:

```
# 列出仓库中所有的软件包
dnf list  == dnf list --all
# 列出已经安装的软件包
dnf list --installed

# 从列出所有安装的包中，查询 vim包
dnf list --installed | grep vim

# 删除 vim 包
dnf remove vim

# 安装vim包，不需要询问， 直接安装：
dnf -y install vim
```

6.2.3 dpkg底层包管理工具

`dpkg` 是 Ubuntu 和 Debian 系统中的底层包管理工具，负责解包和配置 `.deb` 文件。`dpkg` 不会自动解决依赖关系，所以更适合直接管理本地的 `.deb` 包文件。

- 安装 `.deb` 包文件

```
sudo dpkg -i package.deb
```

- 完全卸载 `.deb` 包

```
sudo dpkg -P package.deb
```

- 列出已安装包

```
dpkg -l
```

- 查看已安装包的文件位置

```
dpkg -L package_name
```

6.2.4 apt高级包管理工具

- 更新软件包

```
sudo apt-get update
sudo apt update && sudo apt upgrade -y
```

- 搜索软件包

```
apt search package_name
```

- 安装软件包

```
sudo apt install package_name
```

- 卸载软件包

```
sudo apt remove package_name
```

- 完全卸载包（包括配置文件）

```
sudo apt purge package_name
```

- 列出已安装软件包

```
apt list --installed
```

7. 系统服务管理

7.1 系统服务命令管理

系统服务，即不需要用户独立去安装的软件的服务，而是当系统安装好之后就可以直接使用的服务（内置），如SSH、网络服务等

对于服务管理的核心命令： systemctl

相关操作：

- 1- 显示系统中所有服务

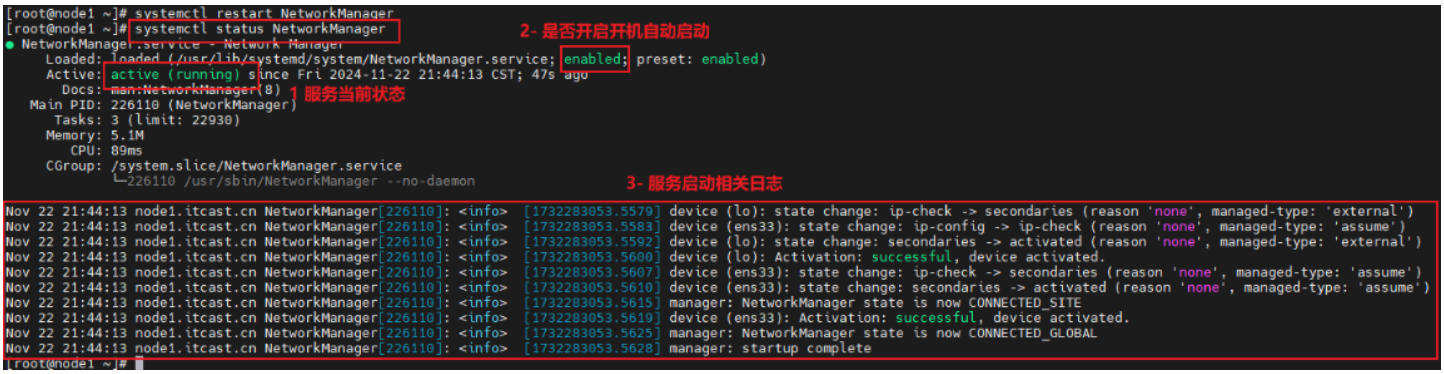
```
systemctl list-units --type service --all（无论启动的没有启动的）
或
systemctl list-units --type service（列出所有启动的服务）
```

- 2- 服务启动|关闭|查看状态|重启

```
核心关键词： start|stop|status|restart

格式： systemctl [选项] 服务名称

# 注意：此操作，会立即生效
```

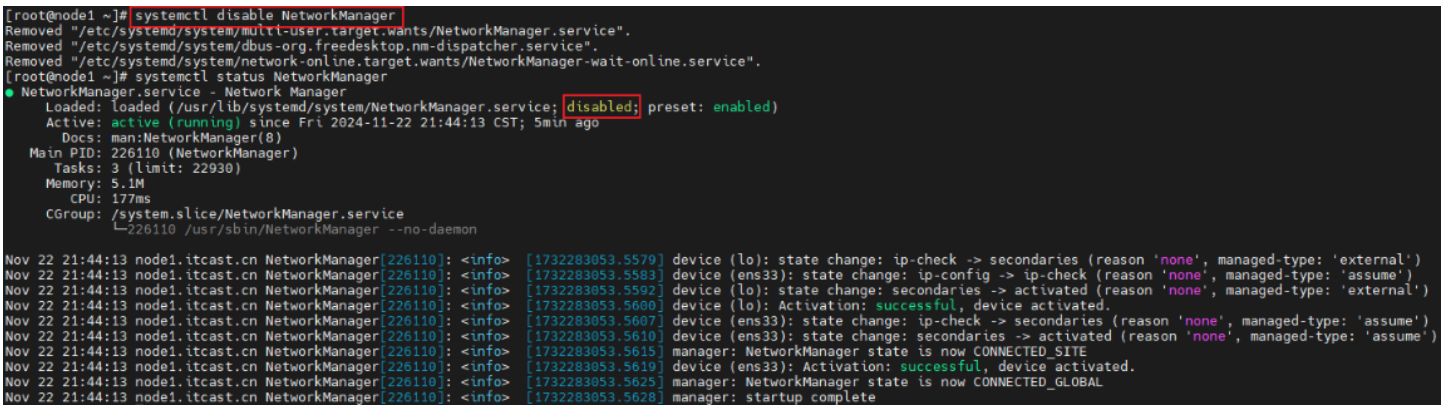


- 3- 服务自启设置：开启|禁用

```
核心关键词：enable|disable
格式： systemctl [选项] 服务名称

关闭自动启动： systemctl disable NetworkManager
开启自动启动： systemctl enable NetworkManager


此操作：下一次启动生效
```



服务日志查询命令

journalctl是一个用于查询和显示系统日志的强大工具。它是 systemd 日志系统的一部分，可以查看系统服务、内核日志以及其他与 systemd 相关的日志信息。

与 systemctl 命令配合使用，可以方便地调试服务问题。

- 查看所有日志:

```
journalctl
```

  - 默认显示所有日志，按时间顺序排列。

- 使用 `Page Up` 和 `Page Down` 键翻页。

- 查看实时日志

```
journalctl -f
```

  - 类似于 `tail -f`，实时显示最新的日志

- 查看某个服务的日志

```
journalctl -u sshd
```

  - 查看 `sshd` 服务的日志。
  - 替换 `sshd` 为其他服务名称（如 `nginx`、`httpd` 等）

- 查看内核日志

```
journalctl -k
```

  - 仅显示与内核相关的日志。

- 基于时间筛选日志

```
查看最近 10 分钟的日志:
journalctl --since "10 minutes ago"

查看某个时间段的日志:
journalctl --since "2023-10-01" --until "2023-10-02"
```

- 查询错误级别的日志

```
journalctl -p err
```

7.2 定时任务

操作系统不可能 24 小时都有人在操作，有些时候想在指定的时间点去执行任务（例如：每天凌晨 2 点去重新启动tomcat），此时不可能真有人每天夜里 2 点去执行命令，这就可以交给计划任务程序

去执行操作了

在Linux中, 使用**crontab命令**来实现定时任务

基本语法：

```
格式: crontab [选项]
-l : list, 列表当前用户下的计划任务程序
-e : edit, 编辑计划任务
```

案例：显示root账号下的所有的计划任务程序

```
crontab -l
```

编辑计划任务程序：

```
crontab -e
```

打开计划任务编辑文件后，可以在此文件中编写我们自定义的计划任务。

计划任务的规则语法格式，以行为单位，一行则为一个计划：

```
分 时 日 月 周 命令的完整路径（绝对路径）

取值范围（常识）：
分：0~59
时：0~23
日：1~31
月：1~12
周：0~7, 0 和 7 表示星期天

四个符号：
*：表示取值范围中的每一个数字
-：做连续区间表达式的，要想表示1~7，则可以写成：1-7
/：表示每多少个，例如：想每 10 分钟一次，则可以在分的位置写：*/10
,：表示多个取值，比如想在 1 点，2 点，6 点执行，则可以在时的位置写：1,2,6
```

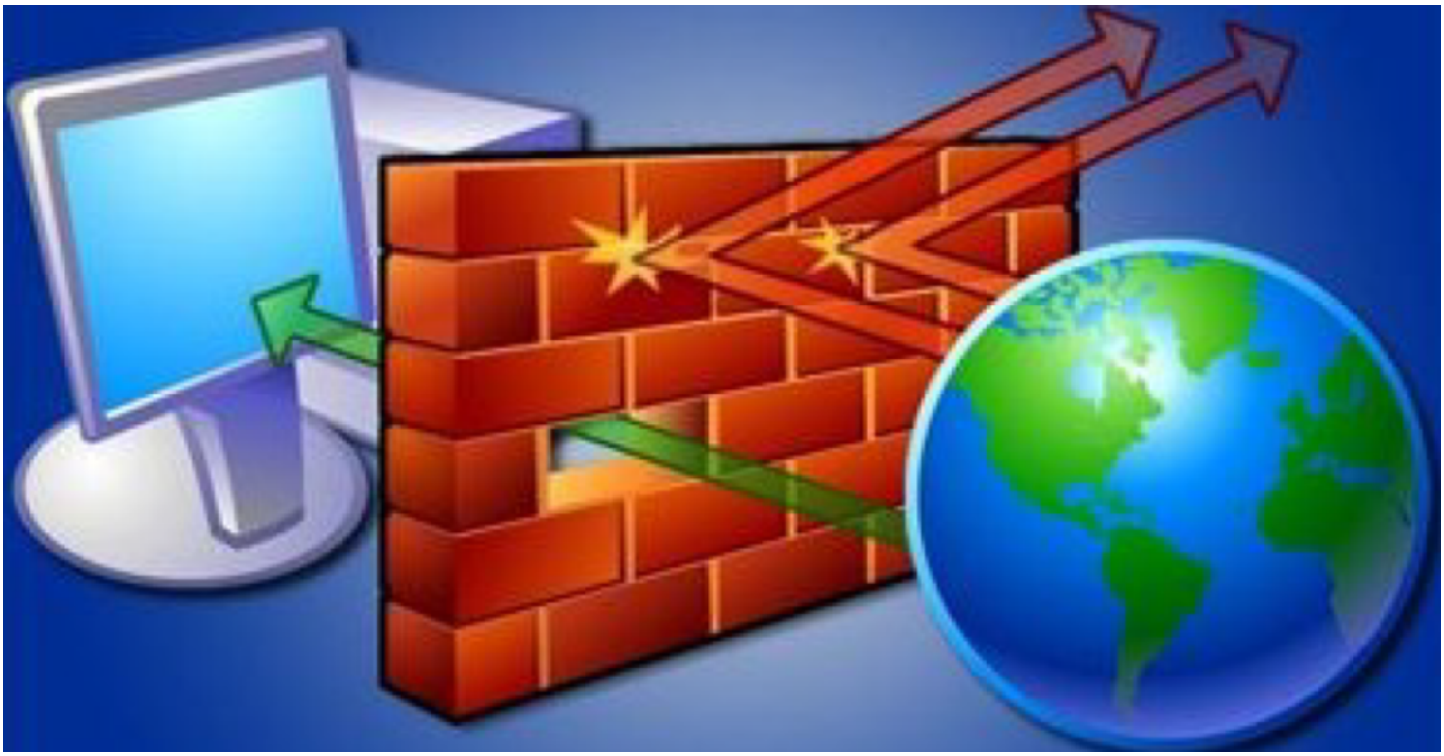
> 注：如何获取命令的完整路径？which或者whereis

> man命令 = manual手册、帮助文档，# man ls

7.3 Linux系统防火墙

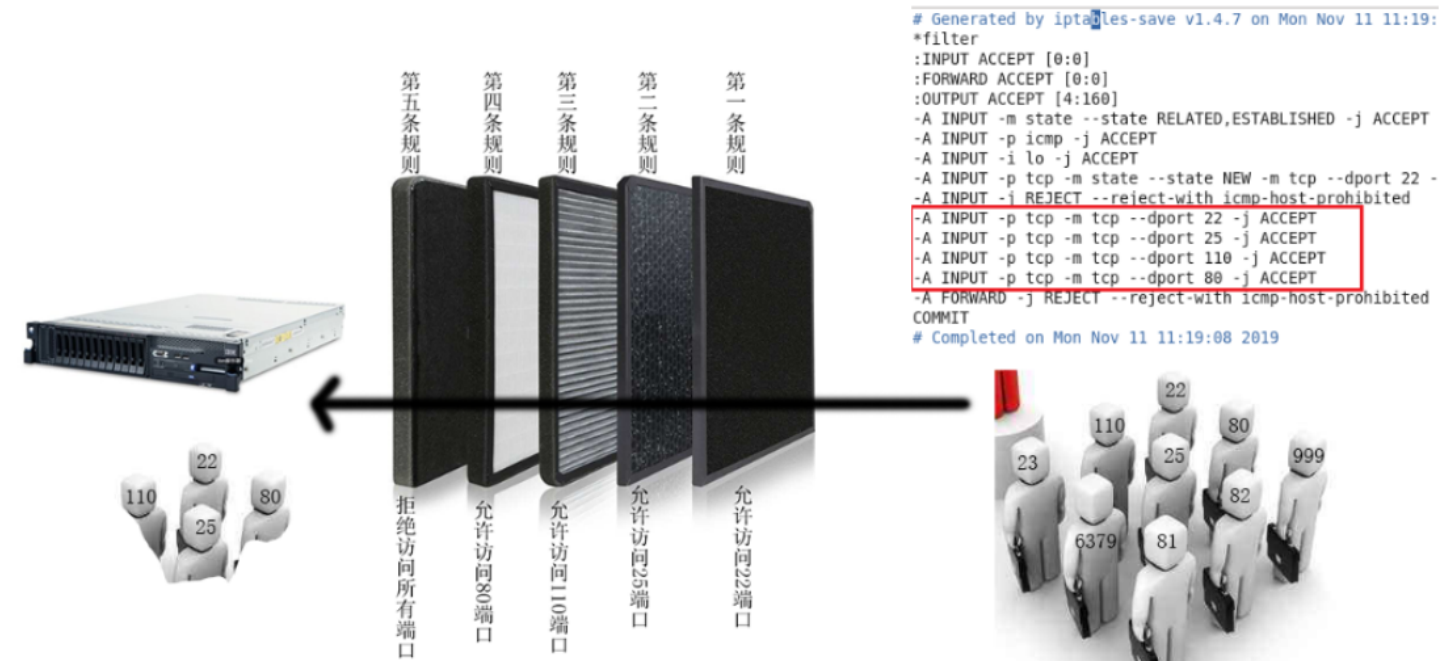
7.3.1 防火墙基本介绍

防火墙：防范一些网络攻击。有软件防火墙、硬件防火墙之分。



防火墙选择让正常请求通过，从而保证网络安全性。

对于Linux防火墙而言, 当网络访问到服务器后， 防火墙会首先判断，该请求对应访问的端口号是否在防火墙中已经被放行， 如果没有， 直接返回拒绝连接， 如果已经放行， 可以访问相关的服务项




```
# Generated by iptables-save v1.4.7 on Mon Nov 11 11:19:
*filter
:INPUT ACCEPT [0:0]
:FORWARD ACCEPT [0:0]
:OUTPUT ACCEPT [4:160]
-A INPUT -m state --state RELATED,ESTABLISHED -j ACCEPT
-A INPUT -p icmp -j ACCEPT
-A INPUT -i lo -j ACCEPT
-A INPUT -p tcp -m state --state NEW -m tcp --dport 22 -
-A INPUT -j REJECT --reject-with icmp-host-prohibited
-A INPUT -p tcp -m tcp --dport 22 -j ACCEPT
-A INPUT -p tcp -m tcp --dport 25 -j ACCEPT
-A INPUT -p tcp -m tcp --dport 110 -j ACCEPT
-A INPUT -p tcp -m tcp --dport 80 -j ACCEPT
-A FORWARD -j REJECT --reject-with icmp-host-prohibited
COMMIT
# Completed on Mon Nov 11 11:19:08 2019
```

7.3.2 防火墙服务管理

查看防火墙状态：

```
systemctl status firewalld
```

启动防火墙：

```
systemctl start firewalld
```

> CentOS6 => iptables，CentOS7 => firewalld

关闭防火墙：

```
systemctl stop firewalld
```

重启防火墙：

```
systemctl restart firewalld
# 或
systemctl reload firewalld
```

> restart与reload有啥区别？答：两者的重启方式不同

restart：先关闭，在开启（先stop，在start）

reload：在不关闭服务的前提下，实际对配置文件的重新加载（不会影响业务）

7.3.3 防火墙配置（规则）

基本语法：

```
1   firewall-cmd [选项1] [选项2] ...
```

查看防火墙的所有区域（预定义规则）

```
1   firewall-cmd --get-zones
```

firewalld增加了区域的概念，所谓区域是指，firewalld **预先准备了几套防火墙策略的集合**，类似于**策略的模板**，用户可以根据需求选择区域。

常见区域及相应策略规则

区域	默认策略
trusted	允许所有数据包
home	拒绝流入的流量，除非与流出的流量相关，允许ssh,mdns,ipp-client,amba-client,dhcpv6-client服务通过
internal	等同于home
work	拒绝流入的流量，除非与流出的流量相关，允许ssh,ipp-client,dhcpv6-client服务通过
public(默认)	拒绝流入的流量，除非与流出的流量相关，允许ssh,dhcpv6-client服务通过
external	拒绝流入的流量，除非与流出的流量相关，允许ssh服务通过
dmz	拒绝流入的流量，除非与流出的流量相关，允许ssh服务通过
block	拒绝流入的流量，除非与流出的流量相关，非法流量采取拒绝操作
drop	拒绝流入的流量，除非与流出的流量相关，非法流量采取丢弃操作

- 查看默认的防火墙区域

```
firewall-cmd --get-default-zone

#显示内容:
public
```

- 列出当前区域有哪些具体的规则

```
firewall-cmd --list-all

# 显示内容:
public (active)
  target: default
  icmp-block-inversion: no
  interfaces: ens33
  sources:
  services: ssh dhcpv6-client
  ports:
  protocols:
  masquerade: no
  forward-ports:
  source-ports:
  icmp-blocks:
  rich rules:


  在其中，目前主要关注 services 和 ports 二部分的内容，分别用于表示当前规则放行的服务或者端口有哪些

  同时也表示，在防火墙配置中，支持端口放行或服务(又或者协议)放行的方式
```

- 添加允许的服务或端口（重点）
  - 添加允许的服务
    - 格式: firewall-cmd --zone=public --add-service=服务名称
    - 案例：把ntp服务添加到防火墙的规则中

```
firewall-cmd --zone=public --add-service=chronyd
```

- 案例：把ftp服务添加到防火墙的规则中

```
firewall-cmd --zone=public --add-service=chronyd
```

- 添加允许的端口号
  - 格式: firewall-cmd --zone=public --add-port=端口号/tcp或udp

```
SSH：22端口、FTP：21端口、Apache：80端口、MySQL：3306端口

tcp协议与udp协议的区别?
答：tcp可靠传输协议（慢），udp不可靠传输协议（快）
```

  - 案例：把80端口/tcp协议添加到防火墙允许的规则中

```
firewall-cmd --zone=public --add-port=80/tcp
```

- 从防火墙规则中移除某个服务或端口
  - 移除服务：firewall-cmd --zone=public --remove-service=ntp
  - 移除端口：firewall-cmd --zone=public --remove-port=80/tcp
- 防火墙规则永久模式（permanent）

```
firewall-cmd --zone=public --add-port=80/tcp --permanent
firewall-cmd --reload        =>      让规则立即生效
```

7.3.4 Ubuntu 防火墙说明

Ubuntu 上默认的防火墙管理工具为ufw

- 查看防火墙状态：查看ufw防火墙的状态， 及其目前放行规则信息

```
sudo ufw status
```

- 开启防火墙： 开启防火墙， 并开启开机自启项

```
sudo ufw enable
```

- 关闭防火墙: 关闭防火墙 并关闭开机自启项目

```
sudo ufw disable
```

7.4 Linux网络服务

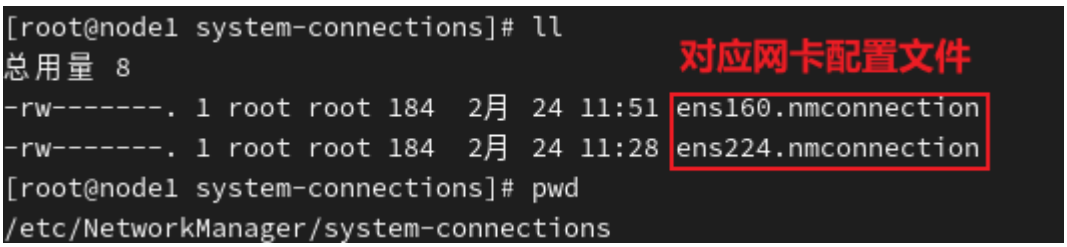
NetworkManager 是 CentOS Stream 9 中默认的网络管理服务，它负责管理网络连接、设备状态、以及网络配置。NetworkManager 支持多种网络类型，包括有线、无线、VPN、移动宽带等。它提供了一个统一的接口来管理网络配置，并且可以通过命令行工具 (`nmcli`)、图形界面 (`nmtui`) 或配置文件来配置网络。

网卡服务管理操作:

- 格式:

```
systemctl [start|stop|restart|enable|disable] NetworkManager
```

网卡对应的配置文件路径: /etc/NetworkManager/system-connections/



配置文件详解:

```
[root@node1 system-connections]# cat ens160.nmconnection
[connection]
id=ens160 连接名称
uuid=21670e0e-0326-41d0-b7a5-54c9366a5556 唯一标识符
type=ethernet 连接类型, 表示以太网
interface-name=ens160 绑定的网络接口名称

[ethernet] 以太网相关配置

[ipv4] ipv4的相关配置
method=auto IPv4 的配置方法为自动（DHCP）
            表示从 DHCP 服务器获取 IP 地址、网关和 DNS。
[ipv6] ipv6的相关配置
addr-gen-mode=default IPv6 地址生成模式为默认
method=auto 表示从网络自动获取 IPv6 地址

[proxy] 代理配置部分
```

7.4.1 网卡相关命令:

- 查看网络接口信息

命令	显示内容
ip addr show 或 ip a	1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000 link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00 inet 127.0.0.1/8 scope host lo valid_lft forever preferred_lft forever inet6 ::1/128 scope host valid_lft forever preferred_lft forever 2: ens160: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000 link/ether 00:0c:29:2e:4e:5c brd ff:ff:ff:ff:ff:ff altname enp3s0 inet 192.168.88.101/24 brd 192.168.88.255 scope global noprefixroute ens160 valid_lft forever preferred_lft forever inet6 fe80::20c:29ff:fe2e:4e5c/64 scope link noprefixroute valid_lft forever preferred_lft forever
nmcli device status	[root@node1 ~]# nmcli device status DEVICE TYPE STATE CONNECTION ens160 ethernet 已连接 ens160 lo loopback 连接（外部） lo

- 启用/禁用网络接口

命令	**显示内容**
启用网卡设备: nmcli device connect 网卡名称	[root@node1 ~]# nmcli device connect ens160 设备 "ens160" 成功以 "e068949b-1fdb-39bf-9a33-633b32092a92" 激活。 激活网卡后, 同时会添加网卡的配置文件
禁用网卡设备: nmcli device disconnect 网卡名称	[root@node1 ~]# nmcli device disconnect ens160 成功断开设备 "ens160"。

- 关闭/启用网络连接

命令	显示内容
启用网络连接 nmcli connection up ens192	[root@node1 system-connections]# nmcli connection down ens160 成功停用连接 "ens160"（D-Bus 活动路径：/org/freedesktop/NetworkManager/ActiveConnection/2）
关闭网络连接 nmcli connection down ens192	[root@node1 system-connections]# nmcli connection up ens160 连接已成功激活（D-Bus 活动路径：/org/freedesktop/NetworkManager/ActiveConnection/5）

- 删除网络连接(会删除网卡配置文件)

命令	显示内容
删除网络设备: nmcli connection delete 网卡名称	[root@node1 system-connections]# nmcli connection delete ens160 成功删除连接 "ens160"（40edfdfd-b6d6-4825-8f95-d9a16120441d）。 删除后, 需要通过重启网卡接口

7.4.2 网卡静态IP配置

配置静态IP是为了确保设备在网络中始终使用固定的IP地址，避免因动态分配（DHCP）导致的IP变化，从而保证服务的稳定性和可访问性。

- 方式一: 通过nmcli命令行完成静态IP配置
  - 格式: nmcli connection modify 网卡名称 ipv4.method manual ipv4.addresses 主机静态IP地址与子网掩码 ipv4.gateway 网关地址 ipv4.dns DNS地址

```
示例:
nmcli connection modify ens160 ipv4.method manual ipv4.addresses 192.168.88.101/24 ipv4.gateway 192.168.88.2 ipv4.dns 8.8.8.8


注意: 设置完, 需要对网卡进行重启
nmcli connection down ens160 && nmcli connection up ens160

&& : 只有前一个命令成功执行（返回状态码为 0），才会执行 && 后面的命令。
```

- 方式二: 通过修改网卡配置文件的方式
  - 步骤一: 打开网卡配置文件

```
vi /etc/NetworkManager/system-connections/ens160.nmconnection

其中:
    /etc/NetworkManager/system-connections/ 用于放置网卡配置文件的目录

    ens160.nmconnection:  指的是ens160这张网卡的配置文件
```

  - 步骤二: 修改或添加以下内容

```
尝试修改配置文件中以下的相关内容:
[ipv4]
method=manual
addresses=192.168.88.101/24
gateway=192.168.88.2
dns=8.8.8.8;

[ipv6]
method=ignore
```

  - 步骤三: 重启网卡

```
#重启网卡服务:
systemctl restart NetworkManager

或者

# 重启网卡
nmcli connection down ens160 && nmcli connection up ens160
```

7.5 Ubuntu Server网络服务

Ubuntu 24.04 默认使用 `systemd-networkd` 作为网络管理服务，负责网络接口的配置和管理。

配置文件位置: /etc/netplan/

- 常用管理命令:

```
sudo networkctl status  # 查看状态
```

```
sudo networkctl reload  # 重启加载
sudo networkctl up  # 启用
sudo networkctl down # 停止

或者
sudo systemctl status systemd-networkd
sudo systemctl start systemd-networkd
....
```

如何修改静态IP地址:

- 1- 修改网卡配置文件

```
cd /etc/netplan/

sudo vim 50-cloud-init.yaml

修改内容如下:
network:
    ethernets:
        eth0:  ## 网卡的名字,  与ip a 看到的网卡名字保持一致（此文本不要粘贴到电脑中）
            dhcp4: false
            addresses:
              - 192.168.88.161/24   ## set static ip
            routes:
              - to: default
                via: 192.168.88.2
            nameservers:
              addresses: [8.8.8.8,8.8.4.4]
    version: 2
```

- 2- 重启网络, 使之生效

```
sudo netplan apply
```

- 3- 测试网络连通性

```
sudo apt install iputils-ping

ping -c 4 www.baidu.com
```

8. 系统核心命令

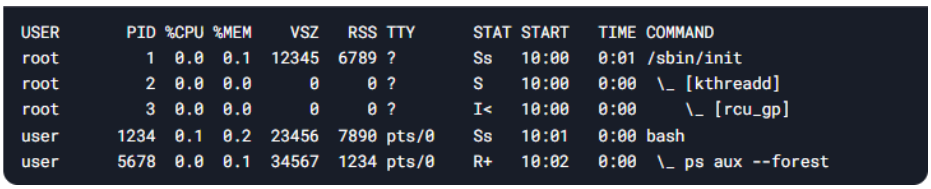
8.1 进程管理命令

什么是进程概念:

> 在 Linux 中，进程是一个正在运行的程序，它包括了代码、数据、打开的文件、内存空间等资源。
>
> 进程 ID（PID）：每个进程都有唯一的进程 ID，用于标识该进程。
>
> 父进程（PPID）：每个进程都有一个父进程。创建新进程的进程称为父进程，新创建的进程称为子进程。
>
> 守护进程（Daemon）：守护进程是一种在后台运行的进程，通常用于提供系统服务，如网络、日志记录等。

- 查看系统进程命令: ps

常用组合	说明	输出结果
aux	查看详细的资源使用情况和进程状态	`USER` : 进程所有者的用户名。 `PID` : 进程 ID。 `%CPU` : CPU 使用率百分比。 `%MEM` : 内存使用率百分比。 `VSZ` : 进程占用虚拟内存大小。 `RSS` : 进程占用物理内存大小。 `TTY` : 终端类型 `STAT` : 进程状态(基本状态 + 附加组成) 基本状态: R(running) : 进程运行 S : 可中断进程(可以随时停止) T(terminate) : 进程被暂停(挂起) ctrl +z D : 不可中断进程(进程正在进行IO读写) Z(zombie) : 僵尸进程，异常的进程 附加组成: s: 进程是控制进程， Ss进程的领导者，父进程/主进程 <: 进程运行在高优先级上， S<优先级较高的进程 N: 进程运行在低优先级上， SN优先级较低的进程

		+: 当前进程运行在前台， R+该表示进程在前台运行 l(小写L): 进程是多线程的， Sl表示进程是以线程方式运行(与程序)使用多线程可以让服务或软件支持更改的访问，但是需要软件支持。 `START` : 进程启动时间 `TIME` : 进程占用的 CPU 时间 `COMMAND` : 启动进程的命令
-ef	查看基本进程信息	`UID` : 进程所有者的用户 ID `PID` : 进程 ID `PPID` : 父进程 ID `C` : CPU 使用率 `STIME` : 进程启动时间 `TTY` : 终端类型 `TIME` : 进程占用的 CPU 时间 `CMD` : 启动进程的命令
--forest	以树状结构显示当前用户的进程	

8.2 杀死进程 kill

格式: kill [选项] [PID]

常用组合	说明
kill <PID>	终止进程
kill -9 <PID>	强制终止进程 **常用信号**: • `SIGTERM` （15）: 正常终止。 • `SIGKILL` （9）: 强制终止
pkill 进程名	终止所有同名进程
pgrep 进程名	查看进程ID

8.3 查询进程所占用端口 ss

格式: ss [选项]

选项	说明
-t	显示 TCP 端口
-u	显示 UDP 端口
-l	显示监听中的端口
-p	显示使用端口的进程信息
-n	以数字形式显示地址和端口（不解析主机名和服务名）
-a	显示所有端口（监听和非监听）

常用组合:

常用组合	说明
ss -tunap	查看所有TCP和UDP端口
ss -tunlp	查看监听中的端口

输出示例:

```
Netid  State   Recv-Q  Send-Q  Local Address:Port   Peer Address:Port  Process
tcp    LISTEN  0       128     0.0.0.0:22          0.0.0.0:*         users:(("sshd",pid=123
4,fd=3))
tcp    ESTAB   0       0       192.168.1.100:22     192.168.1.200:54321 users:(("sshd",pid=5
678,fd=4))
udp    UNCONN  0       0       0.0.0.0:68           0.0.0.0:*         users:(("dhclient",pid
=91011,fd=6))
```

- `Local Address:Port`: 本地地址和端口
- `Peer Address:Port`: 远程地址和端口
- `Process`: 使用该端口的进程名称和 PID

8.4 查看系统状态

8.4.1 top 与 htop 命令

top命令作用： top 命令主要用来对进程以及资源进行监控。

运行结果：

```
top - 09:48:52 up  2:34,  3 users,  load average: 0.00, 0.00, 0.00
Tasks: 180 total,   1 running, 179 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.1 us,  0.1 sy,  0.0 ni, 99.8 id,  0.0 wa,  0.1 hi,  0.0 si,  0.0 st
MiB Mem :   3626.0 total,   3122.8 free,    453.1 used,    275.1 buff/cache
MiB Swap:   4096.0 total,   4096.0 free,      0.0 used.   3172.9 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
    761 root      20   0  168412   8704   7296 S   0.3   0.2   0:08.07 vmtoolsd
      1 root      20   0  170796  12928   9728 S   0.0   0.3   0:01.21 systemd
      2 root      20   0       0      0      0 S   0.0   0.0   0:00.01 kthreadd
      3 root      20   0       0      0      0 S   0.0   0.0   0:00.00 pool_workqueue_
      4 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_g
      5 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_p
      6 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-slub_
      7 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-netns
      9 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/0:0H-events_highpri
     10 root      20   0       0      0      0 I   0.0   0.0   0:00.00 kworker/u512:0-events_unbound
     11 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-mm_pe
     12 root      20   0       0      0      0 I   0.0   0.0   0:00.00 kworker/u512:1-netns
     13 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_kthre
     14 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_rude_
     15 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_trace
     16 root      20   0       0      0      0 S   0.0   0.0   0:00.00 ksoftirqd/0
     17 root      20   0       0      0      0 I   0.0   0.0   0:00.13 rcu_preempt
     18 root      rt   0       0      0      0 S   0.0   0.0   0:00.01 migration/0
     19 root     -51   0       0      0      0 S   0.0   0.0   0:00.00 idle_inject/0
     21 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/0
     22 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/1
     23 root     -51   0       0      0      0 S   0.0   0.0   0:00.00 idle_inject/1
     24 root      rt   0       0      0      0 S   0.0   0.0   0:00.13 migration/1
     25 root      20   0       0      0      0 S   0.0   0.0   0:00.01 ksoftirqd/1
     27 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/1:0H-events_highpri
```

8.4.1.1 top 执行结果说明

第一行是任务队列信息，也可以使用 `uptime命令`

信息	描述
09:48:52	系统当前时间
up xxx days, 2:34,	系统运行时间为 xxx 天 2小时 34 分钟
3 users	当前登录用户数
load average: 0.00, 0.00, 0.00	CPU 负载，即任务队列的平均长度。 三个数值分别为 1 分钟、5 分钟、15 分钟前到现在的平均值。 如何通过top命令查看系统负载情况? > 答：如果你有一个4核心的CPU，而1分钟的负载平均值超过了4，这可能表明系统负载过高。如果负载平均值远高于CPU核心数，这可能表明系统严重过载。

第二行是 `进程数量` 信息

信息	描述
Tasks: 180 total	进程总数

1 running	正在运行的进程数
179 sleeping	睡眠的进程数
0 stopped	停止的进程数
0 zombie	僵尸进程数

第三行是`CPU使用率`信息

信息	**描述**
%Cpu(s): 0.1 us,	用户空间占用 CPU 百分比
0.1 sy	内核空间占用 CPU 百分比
0.0 ni	用户进程空间内改变过优先级的进程占用CPU百分比
99.8 id	空闲 CPU 百分比
0.0 wa	等待输入输出的CPU时间百分比
0.1 hi	hardware IRQ，硬中断占用 CPU 时间百分比
0.0 si	software IRQ，软中断占用CPU的百分比
0.0 st	Steal Time

第四行和第五行显示的是`内存占用`信息

信息	**描述**
KiB Mem : 65633064 total,	物理内存总量
21800724 free,	空闲内存总量
3450376 used,	使用的物理内存总量
40381964 buff/cache	用作内核缓存的内存量
KiB Swap: 8000508 total,	交换区总量
7649788 free,	空闲交换区总量
350720 used.	使用的交换区总量

61509196 avail Mem	当前系统的可用物理内存大小（MiB）。这个值包括空闲内存和一些可以回收的缓存内存

第六行为 `进程自身` 的详细信息

PID	进程 ID 号
USER	进程所有者的用户名
PR	进程优先级
NI	nice 值，负值表示高优先级，正值表示低优先级
VIRT	进程使用的虚拟内存总量，单位kb。VIRT = SWAP + RES
RES	进程使用的、未被换出的物理内存大小，单位 kb。RES = CODE + DATA
SHR	共享内存大小，单位 kb
S	进程状态。D = 不可中断的睡眠状态 R = 运行 S = 睡眠 T = 跟踪/停止 Z = 僵尸进程
%CPU	上次更新到现在的 CPU 时间占用百分比
%MEM	进程使用的物理内存百分比
TIME+	进程使用的 CPU 时间总计，单位 1/100 秒
COMMAND	启动进程对应的命令

8.4.1.2 top 相关使用

快捷方式:

运行 top 之后，可以按下边对应的字母（区分大小写）进行一些功能的切换:

- h 或者?：显示帮助画面，给出一些简短的命令总结说明
- i：忽略闲置和僵死进程。这是一个开关式命令
- q：退出程序
- c：切换显示命令名称和完整命令行。 显示完整的命令。 这个功能很有用
- M：根据驻留内存大小进行排序
- P：根据CPU使用百分比大小进行排序
- T：根据时间/累计时间进行排序

命令行参数:

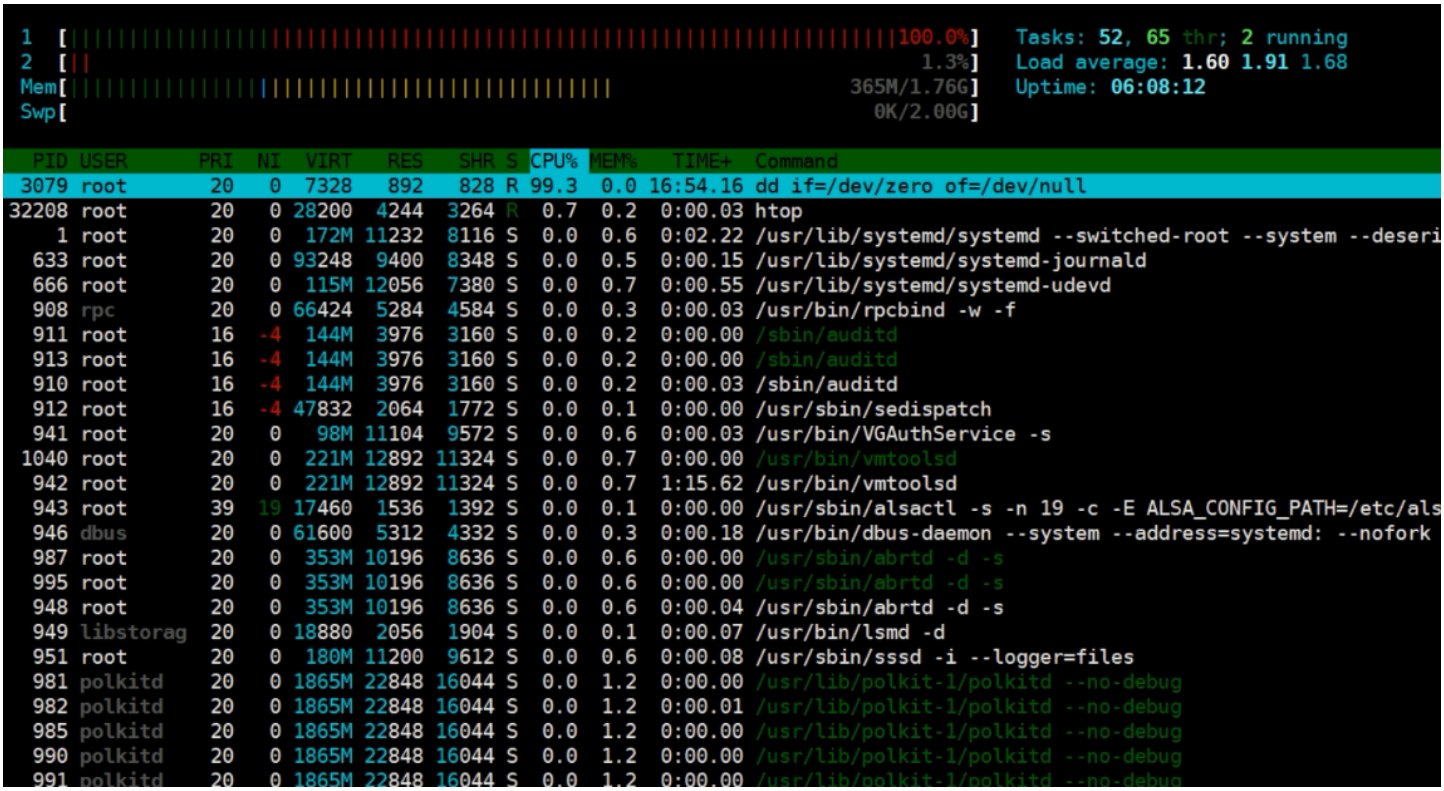
top -h：输出帮助信息

top -u：监控指定用户的进程

top -p：监控指定进程（输入参数为进程pid）

8.4.1.3 htop 命令

htop 命令是增强版的TOP命令，来自EPEL源，比top功能更强



选项:

选项	说明
-d N	指定延迟时间
-u UserName	仅显示指定用户的进程
-s COLUME	以指定字段进行排序

子命令

子命令	说明
s	跟踪选定进程的系统调用

l	显示选定进程打开的文件列表
a	将选定的进程绑定至某指定CPU核心
t	显示进程树

8.4.2 查看磁盘命令 df

作用：disk free，用于查看磁盘的剩余空间。

通常配合 `-h` ，以人性化的方式显示磁盘剩余容量。

案例演示：

```
[root@node1 ~]# df -h
Filesystem                Size  Used Avail Use% Mounted on
devtmpfs                  4.0M     0  4.0M   0% /dev
tmpfs                     1.8G     0  1.8G   0% /dev/shm
tmpfs                     726M  9.1M  717M   2% /run
/dev/mapper/cs_bogon-root  95G  3.1G   92G   4% /
/dev/sda1                 960M  232M  729M  25% /boot
tmpfs                     363M  4.0K  363M   1% /run/user/0
```

8.4.3 查看内存命令 free

作用：用于查看内存的使用剩余空间。

通常配合 `-h` ，以人性化的方式显示内存的剩余空间。

案例演示：

```
[root@node1 ~]# free -h
               total        used        free      shared  buff/cache   available
Mem:           3.5Gi       448Mi       3.1Gi       9.0Mi       277Mi       3.1Gi
Swap:          4.0Gi          0B       4.0Gi
```

8.5 进程挂载后台方案

8.5.1 nohup 命令

nohup

- 作用: 使进程在退出用户登录后继续执行，数据信息和错误默认存储在nohup.out文件中
- 格式: nohup program &

进程的挂起和恢复

Ctrl +Z：挂起

Ctrl +C：终止

```
# 将挂起的进程恢复到前台运行
fg

# 将挂起的进程恢复到后台运行
bg

# 查看被挂起的进程
jobs
```

8.5.2 screen 命令

screen命令相对于nohup是一个更较为稳定的后台运行一些指令。

简易原理：创建screen空间，screen命令维持，在里面运行的命令只要空间在，里面的命令就不会断（后台运行）

```
#1. 安装screen
dnf install -y screen

#2. 运行screen
screen
进入screen虚拟窗口

#3. 执行命令
输入命令 ping baidu.com

#4. 退出screen窗口
退出窗口(异常推荐,正常退出)
ctrl + a 然后 d

#5. 查看screen窗口
查看
screen -ls

#6. 恢复
恢复
screen -r
彻底结束
ctrl + d
```

8.6 主机名配置

主机名（Hostname） 是网络中用于标识一台设备的名称。它可以是简单的名称（如 `server1`），也可以是完整的域名（如 `server1.example.com`）。主机名通常用于：

- 在网络中唯一标识设备
- 方便管理员识别和管理设备
- 在 DNS 解析中与其他设备通信

8.6.1 主机名命名规范

在 Linux 系统中，主机名的命名需要遵循以下规范：

- **长度限制**：主机名最多包含 **253 个字符**。
- **字符限制**：只能使用字母（`a-z`、`A-Z`）、数字（`0-9`）、连字符（`-`）和点号（`.`）。
  - 点号（`.`）通常用于分隔主机名和域名（如 `server1.example.com`）。
- **开头和结尾**：主机名不能以连字符（`-`）或点号（`.`）开头或结尾。
- **大小写**：主机名通常不区分大小写，但建议使用小写字母。
- **避免特殊字符**：不能使用空格、下划线（`_`）或其他特殊字符。

> 注意: 在生产环境中, 主机名需要遵循FQDN格式，以确保网络中的唯一性和系统的正确解析与访问。如果主机名不符合FQDN格式，可能会导致DNS解析问题或其他网络通信问题
>
> FQDN（Fully Qualified Domain Name），即完全限定域名。FQDN按从左到右的顺序排列：主机名 + 域名 + 顶级域名

8.6.2 如何修改主机名

方式一: 使用hostnamectl命令

- 查看当前主机名:

```
hostnamectl
# 或
hostname
```

- 修改主机名:

```
格式: 永久生效
hostnamectl set-hostname new-hostname
```

```
3
4  例如:
5  hostnamectl  set-hostname  node1.itcast.cn
```

方式二: 直接编辑 `/etc/hostname` 文件

- 步骤一: 打开 `/etc/hostname` 文件

```
1  vi /etc/hostname
```

- 步骤二: 修改文件内容为新的主机名
- 步骤三: 重启系统或重新登录系统即可